

local intelligence documentation de

Die Dokumente sind nach Sprache sortiert. Sie enthalten sowohl Hinweise zur aktuellen Implementierung als auch ältere Vorschläge. Ein Vorschlag ist kein Nachweis dafür, dass eine Funktion oder Route existiert. Die Seiten zur MongoDB-Übergabe und zum Projektstart beschreiben die aktuelle Snapshot-Persistenz.

Contents

Einrichtung und Implementierung

- [Projektstart](#)
- [MongoDB-Persistenz: Übergabestand](#)
- [Prototyp: Datenbankverbindung und Datenmodell](#)
- [Beispiele für Datenbankverbindungen \(historisch\)](#)
- [Architektur- und Ablagekonventionen](#)
- [Vorschlag zur Datenlebenszyklusrichtlinie](#)

Datenmodell- und API-Entwürfe

- [Local-Intelligence-Datenmodell v0.1](#)
- [Datenmodellvorschlag v0.1](#)
- [Entitätenkarte v0.1](#)
- [Vorgeschlagene API-Routen](#)
- [Übersicht zu Datenmodellen und API-Routen](#)
- [Ereignistaxonomie und Schema-Governance](#)

Verhaltens- und Simulationsentwürfe

- [Tickbasierte Simulation](#)
- [Intelligentes Signalrouting](#)
- [Autonomiestrategie](#)
- [Beispiel: Signal, Rule und Autonomie](#)
- [Beispiel: Crows und Holy – interaktive Show-Aktivierung](#)

Local Intelligence starten

Diese Anleitung beschreibt den lokalen Betrieb des Prototyps mit MongoDB.

Voraussetzungen

- Node.js 20.19.0 oder neuer und npm
- Docker mit dem dedizierten MongoDB-Container `mon-mongo` oder eine eigene MongoDB-Instanz auf Port 27017
- Die lokalen Ports 27017 und 3000 müssen frei sein

Der vorhandene Container `crowds-mongodb` gehört zum Crowds-System. Diese Anleitung startet, stoppt oder verändert ihn **nicht**. Local Intelligence verwendet `mon-mongo` und eine eigene MongoDB-Datenbank namens `local_intelligence`.

Backend installieren und starten

Im Repository-Hauptordner:

```
cd local-intelligence
npm install
```

Die lokale Datei `.env` enthält die Verbindungszeichenfolge `mongodb://127.0.0.1:27017/local_intelligence`. Kopiere bei einer neuen Arbeitskopie `.env.example` nach `.env` und passe die Werte bei Bedarf an. `.env` enthält lokale Konfiguration und ist von Git ausgeschlossen.

Starte ausschließlich den dedizierten MongoDB-Container:

```
docker start mon-mongo
```

Falls `mon-mongo` nicht existiert, kann stattdessen ein neuer Container mit eigenem persistentem Volume gestartet werden:

```
docker run -d --name local-intelligence-mongo --restart unless-stopped -p 27017:27017 -v local-intelligence-mongo-data:/data/db mongo:7
```

Danach im Verzeichnis `local-intelligence`:

```
npm start
```

Der Server wartet zunächst auf die Datenbank und stellt anschließend gespeicherte Runs wieder her. Bei erfolgreichem Start erscheinen in der Konsole `MongoDB connected` und `Local Intelligence running on port 3000`. Ist MongoDB nicht erreichbar, startet der API-Server nicht. Beende den Server mit **Ctrl+C**; vor dem Herunterfahren werden die Runs erneut gespeichert.

Alternativ lässt sich der Server vom Repository-Hauptordner aus starten:

```
node local-intelligence/app.js
```

Beispieldaten

In der mitgelieferten lokalen `.env` ist `SEED_DEMO=true` gesetzt. Der Server prüft beim Start, ob der Beispielerperimentlauf `experiment-9x9-test` vorhanden ist. Nur wenn er fehlt, wird er über die lokale API angelegt. Wiederholte Starts legen den Beispieldatensatz daher nicht erneut an. Für einen Betrieb ohne Beispieldaten setze `SEED_DEMO=false`. `seedScript2.js` wird nicht automatisch ausgeführt.

Beispielabfragen an die API:

```
GET http://localhost:3000/experiment-runs/experiment-9x9-test/summary
GET http://localhost:3000/agents?experimentRunId=experiment-9x9-test
GET http://localhost:3000/neighborhoods?experimentRunId=experiment-9x9-test
```

Die Oberflächen `script1.html` und `script2.html` im Repository-Hauptordner erwarten die API unter `http://localhost:3000`.

Integrationstest bei erreichbarer MongoDB:

```
npm test
```

Der Test verwendet eine separate Datenbank mit dem Suffix `_test` und löscht sein Testdokument nach Abschluss.

Persistenzumfang und Grenzen

Die vorhandene Simulationslogik verarbeitet einen `ExperimentRun` synchron als Objekt. Um diesen Prototyp mit möglichst geringem Risiko datenbankgestützt zu machen, speichert Local Intelligence jeden vollständigen Run als Snapshot in einem MongoDB-Dokument der Collection `experimentruns` und verwendet dafür die bestehende String-ID. Beim Start werden die Snapshots wieder in den Laufzeit-Cache geladen. API-Änderungen werden vor JSON-Antworten und zusätzlich periodisch gespeichert.

Im aktuellen Laufzeitbetrieb werden Agenten, Nachbarschaften, Signale, Historien, `PropagationEvents` und Metriken noch nicht in normalisierten Collections persistiert. Vorgesehen ist **ein Backend-Serverprozess pro Datenbank**. Ein Run-Snapshot ist auf ungefähr 14 MiB JSON begrenzt; größere Läufe führen beim Speichern zu einem Fehler. Für den Produktiveinsatz oder größere Experimente sollten wachsende Signale, Historien, `PropagationEvents` und Metriken in eigene Collections ausgelagert und der Mehrprozessbetrieb abgesichert werden. Das sind zukünftige Architekturarbeiten und keine Voraussetzung für den lokalen Übergabebetrieb.

Fehlerbehebung

- **MongoDB-Verbindung fehlgeschlagen:** Prüfe `docker ps`, die Erreichbarkeit von Port 27017 und `MONGODB_URI` in `.env`. Verwende nicht `crowds-mongodb` als Ersatz.
- **Port 3000 belegt:** Beende den konkurrierenden Prozess oder passe `PORT` in `.env` an.
- **Port 27017 belegt:** Prüfe mit `docker ps`, ob ein anderer MongoDB-Container den Host-Port belegt. Verändere nicht den Crowds-Container; konfiguriere stattdessen einen eigenen freien MongoDB-Port und passe `MONGODB_URI` an.
- **Abhängigkeiten fehlen:** Führe `npm install` im Verzeichnis `local-intelligence` aus.
- **Seed fehlgeschlagen:** Prüfe die Serverkonsole und ob Port 3000 frei ist. Der Beispiexperimentlauf wird nur angelegt, wenn er noch nicht existiert.

Details zum aktuellen Persistenzdesign stehen in [MongoDB-Persistenz: Übergabestand](#) und in der [Repository-Dokumentation](#).

MongoDB-Persistenz: Übergabestand

Umgesetzter Umfang

- Mongoose verbindet das Backend beim Start über `MONGODB_URI` mit MongoDB. Ist MongoDB nicht erreichbar, startet der HTTP-Server nicht.

- Vorhandene Experimentläufe werden beim Start aus der Collection `experimentruns` in den bestehenden In-Memory-Cache des Simulators geladen.
- Bevor JSON-Antworten der API gesendet werden, werden betroffene Experimentläufe als Snapshots gespeichert. Ein periodischer Flush sichert alle Läufe, die sich aktuell im prozesslokalen Cache befinden; bei regulärem Herunterfahren mit `SIGINT` oder `SIGTERM` werden sie erneut gespeichert.
- Jeder Snapshot verwendet die vorhandene String-ID des Experimentlaufs als `MongoDB-_id`. Dadurch bleiben API- und Seed-IDs erhalten.
- Der Demo-Seed ist optional konfigurierbar und prüft vor dem Anlegen, ob der festgelegte Beispiexperimentlauf bereits existiert.
- `.env` enthält lokale Konfiguration und ist von Git ausgeschlossen. `.env.example` dokumentiert die Konfiguration ohne Geheimnisse.

Bewusste Übergabeentscheidung

Die Simulationslogik wurde in diesem Abschlussdurchgang nicht so umgeschrieben, dass sie asynchrone Änderungen in mehreren Collections vornimmt. Stattdessen wird das vollständige Laufzeitobjekt jedes Laufs im Feld `data` eines MongoDB-Dokuments in `experimentruns` gespeichert. So bleiben die bestehenden synchronen Controller, Regeln und Simulationsticks erhalten; nach einem Neustart werden sie aus MongoDB wiederhergestellt.

Dies ist eine MongoDB-gestützte Persistenz für den aktuellen Prototyp, **keine normalisierte Produktionsarchitektur**. Im aktuellen Laufzeitbetrieb werden Agenten, Nachbarschaften, Signale und Ereignisse noch nicht in normalisierten Collections persistiert. MongoDB begrenzt ein einzelnes BSON-Dokument auf 16 MiB; die Implementierung setzt daher vorsorglich eine Obergrenze von ungefähr 14 MiB JSON pro Run-Snapshot. Vorgesehen ist ein einzelner Backend-Serverprozess pro Datenbank. Da Snapshots periodisch gespeichert werden, können bei einem abrupten Prozess- oder Host-Ausfall die letzten Sekunden an Änderungen verloren gehen.

Nicht Teil des aktuellen Übergabumfangs

- Mehrere API-Instanzen oder ein koordinierter gemeinsamer Cache
- Große Läufe mit unbegrenzt wachsenden Signal-, History- oder Event-Arrays
- Separat persistierte Agenten-, Neighborhood-, Signal- und Event-Dokumente
- Verteilte Transaktionen, Archivierung und Lifecycle-Aufbewahrung
- Session-, Media-, Simulator-, Recovery- und Analytics-Routen aus Entwurfsdokumenten

Empfehlungen für die nächste technische Übergabe

1. Bei wachsenden Datenmengen Historien sowie Propagation- und Protocol-Events in eigene Collections auslagern und den Datenzugriff asynchron gestalten.
2. Schema-Migrationen für Snapshots einführen, bevor sich das Format von `data` inkompatibel ändert; `snapshotVersion` ist dafür reserviert.
3. Vor dem Betrieb mehrerer Backend-Instanzen den In-Memory-Cache entfernen oder durch ein explizites Konsistenz- und Sperrmodell ersetzen.
4. Die Lifecycle-Bereinigung an den tatsächlichen Snapshot-Daten ausrichten und einschließlich abhängiger Daten testen.
5. Vor einem Produktiveinsatz Authentifizierung, Zugriffsschutz, Backups, Monitoring und angemessene MongoDB-Benutzerrechte ergänzen.

Lokaler Betrieb und Prüfung

Siehe [Projektstart](#) für die lokale Einrichtung. Bei laufender MongoDB führt `npm test` im Verzeichnis `local-intelligence` einen Integrationstest in einer separaten Datenbank mit dem Suffix `_test` aus.

Datenbankverbindung und Datenmodell des Prototyps

Hinweis zur Quelle: Die Quelldatei `prototypDatabaseConnectionExplain.md` enthält eine Übersicht des Datenmodells, aber keine Erklärung der Datenbankverbindung. Dieses Dokument korrigiert diese Abweichung und trennt belegtes Prototypverhalten von konzeptionellen oder vorgeschlagenen Entitäten.

1. Aktuelle Datenbankverbindung und Persistenz

Der Prototyp verwendet Mongoose für die Verbindung zu MongoDB. [database.js](#) liest `MONGODB_URI` aus der Umgebung und gibt einen Konfigurationsfehler aus, wenn der Wert fehlt. Die optionale Einstellung `MONGODB_SERVER_SELECTION_TIMEOUT_MS` hat standardmäßig den Wert 10000 Millisekunden.

Beim Start lädt [app.js](#) `local-intelligence/.env`, verbindet sich mit MongoDB, stellt Snapshots der Experimentläufe wieder her und startet anschließend den HTTP-Server. Das In-Memory-Array `store.experimentRuns` dient als Arbeitsspeicher für die Läufe. Der Persistenzdienst speichert jeden Lauf

als data-Snapshot in der Collection `experimentruns` und übernimmt ausgewählte Felder zusätzlich in das Dokument, damit sie abgefragt werden können. API-Antworten, die einem Lauf zugeordnet sind, lösen eine Persistierung aus. Standardmäßig erfolgt außerdem alle 3000 ms ein periodischer Flush; das Intervall lässt sich mit `PERSISTENCE_FLUSH_INTERVAL_MS` konfigurieren. Beim geordneten Herunterfahren werden alle Läufe gespeichert und die Verbindung zu MongoDB geschlossen.

Die Persistenz der Experimentläufe basiert somit auf Snapshots. Dass weitere Mongoose-Model-Schemas existieren, bedeutet nicht, dass aktuell sämtliche Laufzeitdaten über diese Modelle in getrennte Collections geschrieben werden.

2. Ebenen des Datenmodells

Die Quellskizze gruppiert die Modellkonzepte wie folgt:

1. **Experiment:** `ExperimentRun`, `ExperimentParticipant`.
2. **Agenten und Nachbarschaften:** `Agent`, `AgentState`, `AgentStateHistory`, `Neighborhood`, `NeighborhoodRelation`.
3. **Signale und Weiterleitung:** `Signal`, `PropagationEvent`, `Rule`, `RuleSet`, `Threshold`.
4. **Beobachtungen und Ergebnisse:** `Observation`, `ObservationMetric`, `CollectiveBehaviorResult`.
5. **Betrieb und Diagnose:** `ProtocolEvent`, `TechnicalWarning`, `FailureState`.
6. **Unterstützende oder vorgeschlagene Konzepte:** `State`, `Session`, `Media`.

Das Modell der Beziehung heißt in der Implementierung `NeighborhoodRelation`; in der Quelle wird es teilweise `NeighborRelation` genannt.

3. Zentrale konzeptionelle Beziehung

```
ExperimentRun
├─ ExperimentParticipant
├─ Agent — AgentState / AgentStateHistory
│   └─ Neighborhood — NeighborhoodRelation — Agent
│       └─ Signal — PropagationEvent
├─ Observation — ObservationMetric
├─ CollectiveBehaviorResult
├─ ProtocolEvent
├─ TechnicalWarning
└─ FailureState
```

Dies ist ein fachliches Diagramm. Es garantiert nicht, dass jede Verbindung als persistierte Datenbankreferenz oder als 1:n-Beschränkung umgesetzt ist.

4. ExperimentRun und Teilnehmende

ExperimentRun repräsentiert die Ausführung eines Experiments. Sein Schema umfasst den Laufstatus (created, running, paused, finished, failed oder cancelled), Lifecycle-Informationen, die Umgebung (test, rehearsal, research oder production), activeRuleSetId, Statistiken, Diagnosedaten und eine optionale sessionId. Diese sessionId ist eine Kennung für eine externe Session und kein Verweis auf ein lokales Session-Modell. Zur Laufzeit wird der vollständige Experimentlauf als Snapshot in der MongoDB-Collection experimentruns gespeichert. ExperimentParticipant ist als separates Modell definiert, wird vom Snapshot-Persistenzpfad jedoch nicht separat gespeichert.

5. Agenten, Zustände und Nachbarschaften

Agent repräsentiert ein Gerät oder einen Teilnehmenden in der Simulation. Sein Schema umfasst experimentRunId, deviceId, stateId, neighborhoodId, Position, Autonomieeinstellungen, Status (offline, online oder busy) und lastSeen. Der Zustand eines Agenten wird durch AgentState repräsentiert; AgentStateHistory ist für die zeitliche Aufzeichnung von Änderungen vorgesehen. Das Agent-Schema besitzt ein Feld neighborhoodId; die übergreifende Fachregel und das Verhalten der Beziehungen sollten dennoch anhand der jeweiligen Abläufe geprüft werden.

Neighborhood repräsentiert einen lokalen Interaktionskontext. NeighborhoodRelation steht für eine direkte Verbindung zwischen Agenten. Die Quelle beschreibt in der vorgeschlagenen Architektur, dass ein Agent genau einer Nachbarschaft angehört. Diese Aussage ist als fachliche Regel zu prüfen und nicht als durch dieses Diagramm belegte, allgemeine Datenbankbeschränkung zu verstehen.

Beispiel einer lokalen Position:

```
{
  "position": {
    "row": 2,
    "col": 3
  }
}
```

6. Signale, Weiterleitung und Regeln

`Signal` repräsentiert eine Nachricht, die einem Lauf und gegebenenfalls Quell- und Zielagenten zugeordnet ist. Die Quellskizze nennt `sourceAgentId`, `targetAgentId`, `experimentRunId`, `neighborhoodId` und `correlationId` als relevante Beziehungen. `PropagationEvent` dokumentiert einen Weiterleitungs- oder Verarbeitungsschritt und kann auf das `Signal`, beteiligte Agenten, ausgelöste Regeln, einen Status und eine Korrelationskennung verweisen. Diese Liste ist als fachliche Beschreibung zu verstehen; einzelne Felder sollten vor ihrer Verwendung anhand des jeweiligen Schemas geprüft werden.

`Rule` beschreibt eine Verhaltensregel, `RuleSet` fasst Regeln zusammen, und `Threshold` kann eine Aktivierungsschwelle enthalten. `ExperimentRun.activeRuleSetId` kennzeichnet die Referenz auf das aktive Regelset. Ob experimentweite Steuernachrichten und Nachrichten zwischen Agenten dasselbe `Signal`-Modell verwenden sollen, ist weiterhin eine offene Entwurfsfrage.

7. Beobachtungen, Ergebnisse und Diagnosen

`Observation` erfasst ein beobachtetes Ereignis oder eine Situation; `ObservationMetric` speichert zugehörige Messwerte. `CollectiveBehaviorResult` enthält ein Analyseergebnis, etwa Synchronisierung, Weiterleitung, Clusterbildung oder Konsens.

`ProtocolEvent` protokolliert wichtige fachliche oder technische Ereignisse. Die Quellnotizen führen Namen wie `SESSION_CREATED`, `SESSION_CONNECTED`, `SESSION_DISCONNECTED`, `SIGNAL_CREATED`, `SIGNAL_STATUS_CHANGED`, `PROPAGATION_CREATED`, `PROPAGATION_STATUS_CHANGED`, `NEIGHBORHOOD_CREATED`, `NEIGHBORHOOD_UPDATED`, `RULESET_CREATED`, `RULESET_ASSIGNED` und `AGENT_REPLACED` als Beispiele auf. Diese Namen sind nicht alle durch die aktuellen Route-Abläufe bestätigt; `AGENT_REPLACED` kommt beispielsweise im `Neighborhood-Service` vor. `TechnicalWarning` steht für eine technische Warnung, während `FailureState` einen Fehlerzustand repräsentiert. Diese Konzepte erfüllen unterschiedliche Aufgaben und sollten nicht gleichgesetzt werden.

Konzept	Bedeutung
<code>ProtocolEvent</code>	Was ist im System passiert?
<code>TechnicalWarning</code>	Welche technische Warnung oder welches Symptom wurde erkannt?
<code>FailureState</code>	Welcher bestätigte Fehlerzustand liegt vor?

Ein Timeout beim Heartbeat eines Agenten kann beispielsweise eine `TechnicalWarning` erzeugen; ein bestätigter Ausfall kann durch `FailureState` repräsentiert werden; und ein Ersatzvorgang kann als `ProtocolEvent` protokolliert werden. Welche Ereignisse tatsächlich ausgegeben werden, hängt vom implementierten Ablauf ab.

8. Implementierte Modelle und Vorschläge

Der aktuelle `Model-Index` exportiert Schemas für `Agent`, `AgentState`, `AgentStateHistory`, `CollectiveBehaviorResult`, `ExperimentParticipant`, `ExperimentRun`, `FailureState`, `Neighborhood`, `NeighborhoodRelation`, `Observation`, `ObservationMetric`, `PropagationEvent`, `ProtocolEvent`, `Rule`, `RuleSet`, `Signal`, `State`, `TechnicalWarning` und `Threshold`. Damit ist belegt, dass diese Model-Definitionen existieren; daraus folgt nicht, dass jede vorgeschlagene Beziehung oder jeder Ablauf vollständig umgesetzt ist.

Die Quelle führt außerdem `Session` und `Media` auf. Keines der beiden Konzepte erscheint im aktuellen `Model-Index` als eigenes exportiertes Modell. Als vorgeschlagene `Session`-Felder nennt die Quelle `type`, `socketId`, `performerId`, `isConnected`, `lastSeen` und `disconnectReason` sowie einen Lifecycle aus `erstellt`, `verbunden`, `getrennt` und `erneut verbunden`. Dies sind Entwurfsnotizen und keine bestätigten Felder oder Lifecycle-Einträge eines lokalen `Session`-Modells. `Session` und `Media` sind daher als Vorschläge oder externe Konzepte zu behandeln, bis lokale Model-Definitionen und Abläufe ergänzt werden. Auch die Übertragung von Beziehungen auf Ersatzagenten und andere Lifecycle-Abläufe sind als Entwurfsaussagen zu verstehen, sofern ihre Implementierung nicht gesondert belegt ist.

9. Beispiel eines fachlichen Gesamtablaufs

```
ExperimentRun
  ↓
Neighborhood
  ↓
Agent A
  ↓ erzeugt
Signal
  ↓ wird weitergeleitet
PropagationEvent
  ├── Rule / RuleSet
  └── Agent B
      ↓
Observation
  ↓
ObservationMetric
  ↓
CollectiveBehaviorResult
```

Wichtige Systemereignisse können zusätzlich als `ProtocolEvent`-Einträge protokolliert werden. Ein technisches Problem kann eine `TechnicalWarning` auslösen; wird ein Ausfall bestätigt, kann außerdem ein `FailureState` erfasst werden.

Eine kompakte Übersicht bietet die [Entitätenkarte](#). Ein konkretes, ausdrücklich beispielhaftes Signal-/Autonomieszenario steht unter [Beispiel: Signal, Autonomie und Folgeaktion](#).

Datenbankverbindungen

Status: Die folgenden Beispiele sind ältere Entwurfsskizzen für eine normalisierte MongoDB-Implementierung und entsprechen **nicht** der aktuellen API-Implementierung. Local Intelligence persistiert derzeit vollständige `ExperimentRun`-Snapshots. Der implementierte Persistenzpfad ist in der [aktuellen Architekturübersicht](#) und im [Run-Persistenzadapter](#) beschrieben. In der aktuellen API müssen beim Anlegen eines Agenten `row` und `col` angegeben werden. Das folgende Beispiel ist historisch und darf nicht als aktueller API-Vertrag verstanden werden.

Agenten

Agent registrieren

```
/* REGISTER AGENT */
router.post("/agents", async (req, res) => {
  try {
    const agent = await Agent.create({
      deviceId: req.body.deviceId,

      // Initial state
      stateId: null,

      // Agent is not assigned to a neighborhood yet
      neighborhoodId: null,

      // Position will be assigned later
      position: {
        row: req.body.row ?? null,
        col: req.body.col ?? null,
      },

      // Device information
      deviceInfo: {
        model: req.body.deviceInfo?.model ?? null,
        platform: req.body.deviceInfo?.platform ?? null,
        version: req.body.deviceInfo?.version ?? null,
      },
    },
```

```

        direction: req.body.direction ?? 0,

        status: "online",

        lastSeen: new Date(),

        metadata: req.body.metadata ?? {},
    });

    res.status(201).json(agent);

} catch (error) {
    // Duplicate deviceId
    if (error.code === 11000) {
        return res.status(409).json({
            error: "Agent with this deviceId already exists",
        });
    }

    res.status(500).json({
        error: "Failed to register agent",
        details: error.message,
    });
}
});

```

Zustand eines Agenten aktualisieren

```

router.patch("/agents/:id/state", async (req, res) => {
    try {
        const agent = await Agent.findById(req.params.id);

        if (!agent) {
            return res.status(404).json({
                error: "Agent not found",
            });
        }

        const previousState = agent.stateId;

        agent.stateId = req.body.stateId;
        agent.lastSeen = new Date();

        await agent.save();

        res.json(agent);

    } catch (error) {
        if (error.name === "CastError") {
            return res.status(400).json({
                error: "Invalid agent ID",
            });
        }
    }
});

```

```

    }

    res.status(500).json({
      error: "Failed to update agent state",
      details: error.message,
    });
  }
});

```

Die Codebeispiele behalten absichtlich ihren ursprünglichen Entwurf für ein normalisiertes Datenmodell bei. Sie sind Referenzskizzen und kein Beleg dafür, dass diese Endpunkte oder separate MongoDB-Dokumente pro Agent in der aktuellen Local-Intelligence-API implementiert sind. Zur Lebenszyklusrichtlinie und ihrem Implementierungsstatus siehe die [Richtlinie zum Datenlebenszyklus](#).

Architektur- und Ablagekonventionen

Dieses Dokument beschreibt die aktuelle Architektur des Local-Intelligence-Prototyps und legt fest, wo neue Dateien hingehören. Es führt das aktuelle MongoDB-Snapshot-Persistenzdesign und die bestehenden Architekturkonventionen zusammen.

Grundregel

`routes/` und `controllers/` sind HTTP-nahe Schichten. Fachlogik gehört nach `domain/` oder `services/`. Der Ordner `middlewares/` ist echten Express-Middlewares vorbehalten, typischerweise Funktionen mit der Signatur `(req, res, next)`.

Aktuelle Struktur

<code>local-intelligence/</code>	
<code>routes/</code>	Express-Routen und Handler-Zuordnungen
<code>controllers/</code>	HTTP-Handler und Request-/Response-Adapter
<code>repositories/</code>	In-Memory-Run-Zugriff und fachliche Suchhilfen
<code>constants/</code>	Fachliche Zustands-, Aktions-, Propagations- und Statuswerte
<code>domain/</code>	
<code>rules/</code>	Regelauswertung und Regelausführung
<code>signals/</code>	Signalverarbeitung und Propagation
<code>behavior/</code>	Agenten- und Schwarmverhalten ohne HTTP-Abhängigkeit
<code>services/</code>	
<code>autonomy/</code>	Autonomieentscheidungen und Aktionen pro Agent
<code>simulation/</code>	Simulationsticks und wiederholte Simulationsläufe
<code>analysis/</code>	Agenten-, Signal-, Nachbarschafts- und Kollektivanalyse

logging/ Fehlerereignissen	Protokollierung von Zustands-, Propagations-, Warnungs- und
scheduling/	Zustandsabhängige und verzögerte Aktionen
utils/	Kleine technische Hilfsfunktionen ohne Fachworkflow
models/	Mongoose-Modelldefinitionen
config/	Konfiguration der Datenbankverbindung

Verantwortlichkeiten

routes/

Routen legen HTTP-Methode, Pfad und Handler-Zuordnung fest. Sie sollen weder den Store durchsuchen noch den Fachzustand verändern oder Simulationslogik ausführen.

```
router.post("/rules", createRule);
```

controllers/

Controller lesen Request-Daten, rufen Repositories, Domain-Funktionen oder Services auf und erstellen HTTP-Antworten. Der aktuelle Simulator verändert ein prozesslokales Experiment-Run-Objekt. Persistenz-Middleware speichert den betroffenen Run-Snapshot in MongoDB, bevor sie eine JSON-Antwort sendet.

repositories/

Repositories kapseln Suchvorgänge und Mutationen innerhalb der bestehenden In-Memory-Run-Strukturen. Sie verwenden den beim Start wiederhergestellten `store.js`-Cache; die MongoDB-Persistenz wird separat durch `services/runPersistence.js` verwaltet:

```
MongoDB-Experiment-Run-Snapshots
-> Wiederherstellung beim Start
-> prozesslokaler store.js-Cache
-> Controller / Domain / Services
    -> Snapshot-Schreibvorgang bei JSON-Antworten
    -> periodischer Snapshot-Flush
```

Zu den aktuellen Repository-Modulen gehören:

```
repositories/
  experimentRunRepository.js
  agentRepository.js
  neighborhoodRepository.js
  rulesetRepository.js
```

```
ruleRepository.js  
signalRepository.js
```

Der Persistenzadapter speichert pro MongoDB-Dokument einen serialisierten vollständigen Experiment-Run unter Verwendung der bestehenden String-ID. Das ist ein Übergabedesign für den Prototyp, keine normalisierte Persistenz separater Agent-, Neighborhood-, Signal- oder Ereignisdokumente. Der Cache ist prozesslokal; die aktuelle Konfiguration ist für einen Backend-Prozess pro Datenbank vorgesehen. Für Snapshot-Daten gilt eine Sicherheitsgrenze von 14 MiB JSON.

Repositories enthalten ausschließlich Datenzugriff. Regelentscheidungen, Signalpropagation und Simulation gehören nach `domain/` oder `services/`. Siehe die [Repository- und Persistenzhinweise](#) und den [Run-Persistenzadapter](#).

domain/

Hier liegt der fachliche Kern der lokalen Intelligenz:

- `domain/rules/`: bestimmt, welche Regeln gelten und welche Aktionen sie auslösen.
- `domain/signals/`: verarbeitet Signale und legt Propagationsverhalten und -reichweite fest.
- `domain/behavior/`: implementiert Agenten- und Schwarmverhalten ohne HTTP-Bezug.

Die Domain darf technische Services wie Logging verwenden, aber keine Express-Routen importieren.

services/

Services koordinieren größere Abläufe:

- `autonomy/`: Entscheidungen pro Agent und autonome Aktionen.
- `simulation/`: einzelne Ticks und wiederholte Simulationsläufe.
- `analysis/`: Messung und Interpretation von Agenten-, Signal-, Nachbarschafts- und Zustandsverteilungen.
- `logging/`: einheitliche Aufzeichnung von Laufzeitereignissen in einem Experiment-Run.
- `scheduling/`: Planung und Verarbeitung zustandsabhängiger Aktionen.

utils/

Nur kleine, möglichst zustandslose Hilfsfunktionen gehören hierher, zum Beispiel geometrische Berechnungen in `utils/geometry.js`.

constants/

Wiederverwendbare fachliche Werte gehören hierher, damit der Code nicht von schwer lesbaren magischen Zahlen oder wiederholten Status-Strings abhängt:

```
constants/  
  actions.js  
  statuses.js  
  propagation.js
```

State-IDs werden weiterhin mithilfe von `stateHelpers.js` anhand der Definitionen in `store.js` aufgelöst. Die bestehende Store-Struktur bleibt unverändert.

Wichtige Abhängigkeiten

Der zentrale Simulations- und Signalverarbeitungsablauf ist:

```
Simulationstick  
-> Autonomieentscheidungen und Signalauswertung  
-> Signalerstellung / Signal-Engine  
    -> Nachbarschaftsanalyse  
    -> Regelauswertung und -ausführung  
        -> Verhalten und Zustandsänderungen  
        -> Propagation  
-> Ereignisprotokollierung
```

`services/analysis/analyzeNeighborhood.js` wird sowohl von der Autonomie als auch von der Signal-Engine verwendet und gehört deshalb nach `analysis/`.

Signalverarbeitungsablauf

Ein Signal durchläuft grundsätzlich diese Schritte:

```
Signalerstellung  
-> processSignal  
-> Nachbarschaftsanalyse  
-> Regelauswertung  
-> Regelausführung  
-> Zustandsänderung  
-> optionale Propagation  
-> Protokollierung
```

Konkret:

1. `createSignal.js` validiert Quelle und Ziel, erstellt das Signal und fügt es dem aktuellen Run hinzu.
2. `signalEngine.js` setzt den Signalstatus auf `processing` und beginnt mit der Verarbeitung.
3. `analyzeNeighborhood.js` ermittelt lokale Agenten- und Zustandsinformationen.
4. `evaluateRules.js` prüft das aktive `RuleSet`, den `Scope`, den `Threshold` und Nachbarschaftsbedingungen.
5. `executeRule.js` führt eine passende Aktion aus, zum Beispiel Aktivierung, Blockierung, Synchronisierung oder Propagation.
6. Betroffene Agenten und das Signal erhalten aktualisierte Zustands- oder Statuswerte.
7. Bei der Propagation werden neue Signale mit verringertem TTL und erhöhtem HopCount erstellt.
8. `runLogger.js` protokolliert Zustands-, Propagations-, Warnungs- und Fehlerereignisse im Run.

Zustands-Trigger und verzögerte Signale

Regeln können auf Zustandsänderungen reagieren. Die Bedingung steht unter `trigger`, die Aktion unter `action` und ein später zu sendendes Signal unter `appendedSignal`:

```
{
  enabled: true,
  scope: "agent",
  agentId: "agent-a",
  trigger: {
    type: "state_changed",
    fromState: "inactive",
    toState: "active",
  },
  action: "send_signal",
  appendedSignal: {
    delayMs: 5000,
    targetAgentId: "agent-x",
    signalType: "follow_up",
    signalPayload: {
      strength: 1,
    },
    signalProperties: {
      propagationMode: "unicast",
      propagationScope: "neighborhood",
    },
  },
},
}
```

Der Ablauf:

```
Zustandsänderung
-> state_changed-Trigger auswerten
```

```
-> appendedSignal als ausstehend speichern  
-> bis zu einem Simulationsttick warten  
-> createSignal aufrufen, sobald executeAt erreicht ist
```

Es wird kein unkontrolliertes `setTimeout` verwendet. Ausstehende Aktionen werden in `run.scheduledActions` gespeichert und von `simulationTick.js` verarbeitet. `delayMs` misst daher verstrichene Echtzeit; die Ausführung erfolgt jedoch erst in einem Simulationsttick, nachdem die Aktion fällig ist.

Importregeln

- Routen importieren Controller.
- Controller importieren Repositories, Domain-Funktionen und Services.
- Direkte `store`-Imports gehören in Repositories oder in die ausdrücklich dafür vorgesehene Store-Schicht.
- Domain-Code darf Services und Utils importieren, aber keine Routen.
- Services dürfen Domain-Code, andere Services und Utils importieren.
- Neue zyklische Abhängigkeiten vermeiden.
- Relative Imports immer vom Speicherort der importierenden Datei aus berechnen.
- Nach dem Verschieben von Dateien alle statischen und dynamischen Imports prüfen.

Wohin gehört eine neue Datei?

- Verarbeitet sie `req`, `res` oder `next`? Nach `routes/` oder `controllers/`.
- Entscheidet sie fachlich über Regeln, Signale oder Verhalten? Nach `domain/`.
- Koordiniert sie mehrere fachliche Schritte oder Simulationstticks? Nach `services/`.
- Führt sie eine kleine, zustandslose Berechnung aus? Nach `utils/`.
- Ist sie Express-Querschnittsinfrastruktur wie Authentifizierung oder Fehlerbehandlung? Nach `middlewares/`.

Namenskonventionen

Dateinamen sollen beschreibendes `camelCase` verwenden, zum Beispiel `signalEngine.js`, `runLogger.js` und `geometry.js`. Abkürzungen und unklare Namen vermeiden.

Validierung nach Strukturänderungen

Mindestens diese Prüfungen aus dem Repository-Hauptverzeichnis ausführen:

```
find local-intelligence -name '*.js' -print0 | xargs -0 -n1 node --check  
node -e "import('./local-intelligence/app.js')"
```

Einen Startcheck nur dann ausführen, wenn der Serverprozess anschließend bewusst beendet oder separat verwaltet wird. Beim aktuellen datenbankgestützten Backend ist der Import von `app.js` vom tatsächlichen Serverstart und der MongoDB-Verbindung zu unterscheiden.

Verwandte Dokumente: [Richtlinie zum Datenlebenszyklus](#) und [Beispiele zum Datenbankverbindungsentwurf](#).

Richtlinie zum Datenlebenszyklus v0.1

1. Zweck und Implementierungsstatus

Dieses Dokument beschreibt eine vorgeschlagene Aufbewahrungs- und Lebenszyklusstrategie für Betriebs-, Experiment-, Forschungs-, Proben-, Test- und Produktionsdaten von Crowds. Ziel ist, Daten nur so lange wie nötig aufzubewahren und zugleich wichtige Forschungs- und Betriebsinformationen zu erhalten.

Die Richtlinie beschreibt das angestrebte Verhalten. Sie wird **für den aktuellen MongoDB-Snapshot-Speicher von Local Intelligence nicht automatisch durchgesetzt**. Die aktuelle Anwendung speichert vollständige Experiment-Run-Snapshots in der Collection `experimentruns`. Es gibt Bereinigungsshelfer, diese arbeiten jedoch mit separaten Mongoose-Collections und sind weder an den Snapshot-Speicher angebunden noch über die aktuelle API erreichbar. Auch die vorgeschlagene Session-TTL ist nicht aktiv. Siehe den [aktuellen Run-Persistenzadapter](#), die [Konfiguration der Lebenszyklusrichtlinie](#) und die [Architekturübersicht](#).

Die Richtlinie umfasst:

- Protokollereignisse
- Session-Daten
- Geräte- und Agentendaten
- Experiment-Runs
- Propagationsereignisse
- Signale
- Beobachtungen

- Analytics-Zusammenfassungen
- Simulatorengaben
- Testdaten
- Probanden
- Operative Produktionsdaten

2. Experimentumgebungen

Jeder Experiment-Run sollte seine Umgebung angeben. Unterstützte Werte:

- test
- rehearsal
- research
- production

Die Umgebung bestimmt die vorgesehene Aufbewahrungs- und Bereinigungsrichtlinie.

3. Richtlinie zum Datenlebenszyklus

Umgebung	Vorgesehene Aufbewahrung	Rohdaten	Zusammenfassung	Vorgesehene Maßnahme
Test	7 Tage	Löschen	Nicht erforderlich	Experiment und abhängige Daten löschen
Probe	30 Tage	Löschen	Wichtige Zusammenfassungen behalten	Rohdaten löschen, Zusammenfassungen behalten
Forschung	Langfristig; die Richtlinienkonfiguration legt derzeit 3.650 Tage fest	Archivieren	Langfristig aufbewahren	Für Analysen archivieren
Produktion	Betriebliche Richtlinie	Nach Bedarf aufbewahren	Aufbewahren	Gemäß betrieblichen Anforderungen

aufbewahren

Diese Werte sind Richtlinienziele. Sie bedeuten nicht, dass Local Intelligence diese Fristen derzeit auf seine Snapshots anwendet.

4. Testdaten

Testdaten sind temporär. Laut Richtlinie dürfen abgeschlossene, fehlgeschlagene oder abgebrochene Testexperimente nach sieben Tagen gelöscht werden. Zu den abhängigen Rohdaten gehören:

- ExperimentParticipant
- Signal
- PropagationEvent
- Observation
- ProtocolEvent
- TechnicalWarning
- FailureState

Anschließend wird der zugehörige ExperimentRun gelöscht. Laufende Experimente dürfen niemals automatisch gelöscht werden. Die Testbereinigung darf keine Forschungs- oder Produktionsexperimente löschen.

5. Probendaten

Probendaten entstehen bei Vorbereitung und Systemvalidierung vor echten Betriebsereignissen. Rohe Probendaten dürfen nach 30 Tagen gelöscht werden. Wichtige Experimentzusammenfassungen, Ergebnisse und Kennzahlen sollten für betriebliche Prüfungen und spätere Vergleiche erhalten bleiben.

6. Forschungsdaten

Forschungsexperimente haben langfristigen Wert. Aufzubewahren sind:

- Experimentmetadaten
- Experimentzusammenfassungen und -ergebnisse

- Experimentstatistiken
- Wichtige Analytics-Zusammenfassungen
- Relevante Protokollereignisse
- Relevante Propagationsdaten

Rohe Forschungsdaten sollten nach Möglichkeit archiviert und nicht dauerhaft gelöscht werden. Die aktuelle Richtlinienkonfiguration legt eine nominelle Aufbewahrungsdauer von 3.650 Tagen und die Aktion `archive` fest. Diese Konfiguration ist für sich genommen kein aktiver Archivierungsjob.

7. Operative Produktionsdaten

Produktionsdaten werden entsprechend betrieblichen und kommerziellen Anforderungen aufbewahrt. Die Testbereinigung darf keine Produktionsdaten löschen. Geräteinformationen sind auf die für Systembetrieb, Experimentzuordnung und Diagnose erforderlichen Felder zu beschränken.

8. Session-Daten

Die Richtlinie sieht vor, getrennte Sessions als temporäre Betriebsdaten zu behandeln und nach sieben Tagen zu entfernen; aktive Sessions sollen bestehen bleiben. **Diese Session-TTL ist in der aktuellen Local-Intelligence-API nicht implementiert:** Es gibt weder ein lokales Session-Modell noch einen TTL-Index. Ein optionales externes Feld `sessionId` an einem Experiment-Run bedeutet nicht, dass lokale Session-Datensätze oder eine TTL-Bereinigung existieren.

9. Protokollereignisse

Protokollereignisse sind nach ihrem Zweck zu klassifizieren. Für Laufzeitereignisse können kürzere Aufbewahrungsfristen gelten. Ereignisse zu Forschungs- oder Produktionsexperimenten sind aufzubewahren oder zu archivieren, wenn sie benötigt werden für:

- Rekonstruktion von Experimenten
- Debugging
- Analytics
- Audits
- Forschung

10. Propagationseignisse

Propagationseignisse bilden die Signalweitergabe zwischen Agenten ab und gelten als rohe Betriebs- oder Experimentdaten. Ihr vorgesehener Lebenszyklus hängt von der Experimentumgebung ab:

- Test: nach Ablauf der Testaufbewahrungsfrist löschen
- Probe: nach Ablauf der Aufbewahrungsfrist für Proben löschen
- Forschung: für langfristige Analysen archivieren
- Produktion: gemäß betrieblichen Anforderungen aufbewahren

11. Minimierung von Gerätedaten

Agenten- und Gerätedaten sind zu minimieren. Es dürfen nur Informationen gespeichert werden, die erforderlich sind für:

- Agentenidentifikation
- Kommunikation
- Positionsverfolgung
- Richtungsbehandlung
- Experimentzuordnung
- Betriebsdiagnose

Nicht benötigte Gerätemetadaten dürfen nicht erhoben werden.

12. Bereinigungsverfahren und aktuelle Einschränkungen

Die Richtlinie beschreibt die folgenden Endpunktformen:

Manuelle Testbereinigung (vorgeschlagen)

```
DELETE /experiments/:id/test-run
```

Dieser vorgeschlagene Endpunkt würde ein abgeschlossenes Testexperiment samt abhängigen Rohdaten löschen.

Automatische Testbereinigung (vorgeschlagen)

```
POST /lifecycle/cleanup/test-experiments
```

Dieser vorgeschlagene Prozess würde Testexperimente entfernen, deren Aufbewahrungsfrist abgelaufen ist. Er wäre beschränkt auf:

```
environment = test
status = finished | failed | cancelled
```

Laufende Experimente sind ausgeschlossen. Diese Lebenszyklus-Routen gehören nicht zur aktuellen API. Die Bereinigungshelfer im Repository fragen normalisierte Mongoose-Modelle und -Collections ab; sie bereinigen nicht die vollständigen Run-Snapshots der aktuellen Anwendung. Außerdem weist der manuelle Helfer nur den Status `running` zurück, statt die vollständige obige Liste abgeschlossener Statuswerte durchzusetzen. Diese Helfer dürfen daher nicht als Durchsetzung der Richtlinie für aktuelle Local-Intelligence-Daten betrachtet werden.

13. Datensicherheitsregeln

1. Die Testbereinigung darf niemals Forschungsexperimente löschen.
2. Die Testbereinigung darf niemals Produktionsexperimente löschen.
3. Laufende Experimente dürfen niemals automatisch gelöscht werden.
4. Forschungszusammenfassungen sollen langfristig erhalten bleiben.
5. Wichtige Zusammenfassungen von Proben sollen auch nach dem Löschen der Rohdaten verfügbar bleiben.
6. Gerätedaten sind zu minimieren.
7. Daten mit langfristigem Forschungswert sind nach Möglichkeit zu archivieren statt zu löschen.
8. Bereinigungsvorgänge sollen in Produktionsumgebungen protokolliert werden.

14. Ablauf des Datenlebenszyklus





15. Aktuelle Implementierung

Das Repository enthält folgende Bausteine für Richtlinie und Services:

- `ExperimentRun.environment` kennzeichnet die Datenumgebung im Mongoose-Modell und in den Metadaten des persistierten Snapshots.
- `dataLifecyclePolicy.js` definiert Richtlinienwerte, darunter 3.650 Tage für Forschung.
- `dataLifecycleService.js` enthält einen manuellen Bereinigungshelfer für normalisierte Mongoose-Collections.
- `cleanupTestExperiments.js` enthält einen Helfer zur automatischen Bereinigung abgelaufener Testexperimente.
- Eine MongoDB-TTL-Richtlinie für getrennte Session-Datensätze ist lediglich vorgeschlagen; in der aktuellen Local-Intelligence-API gibt es kein lokales Session-Modell und keinen aktiven TTL-Index.

Implementierungsstatus von Local Intelligence: Die Anwendung speichert vollständige Experiment-Run-Snapshots in `experimentruns`. Die genannten Bereinigungshelfer sind nicht mit diesem Snapshot-Speicher verbunden; die vorgeschlagenen Lebenszyklus-Routen sind in der aktuellen API nicht eingebunden. Die Richtlinie darf für Local-Intelligence-Daten nicht als automatisch durchgesetzt gelten. Der [Run-Persistenzadapter](#) zeigt den aktuellen Persistenzpfad.

Das angestrebte Design trennt die Lebenszyklusrichtlinie von einzelnen Fachrouten, damit zukünftige Aufbewahrungsregeln ergänzt werden können, ohne jedes Datenmodell ändern zu müssen.

16. Zukünftige Verbesserungen

Für zukünftige Versionen sind folgende Verbesserungen vorgesehen:

- Geplante automatische Bereinigungsjobs
- Separate Bereinigung für Probanddaten
- Datenarchivierung

- Export vor dem Löschen
- Datenanonymisierung
- Audit-Protokolle für Bereinigungen
- Konfigurierbare Aufbewahrungsfristen
- Aufbewahrungsrichtlinien für Produktionsdaten
- Speicherung archivierter Forschungsdatensätze
- Lebenszyklusüberwachung und -metriken

17. Zusammenfassung

Die vorgeschlagene Crowds-Datenlebenszyklusstrategie unterscheidet zwischen temporären Betriebsdaten und langfristigen Forschungs- beziehungsweise Produktionsdaten. Testdaten sollen nach einer festgelegten Frist gelöscht werden. Rohe Probanddaten können gelöscht werden, während wichtige Zusammenfassungen erhalten bleiben. Forschungsdaten sollen langfristig archiviert werden. Produktionsdaten werden gemäß betrieblichen Anforderungen aufbewahrt.

Die Strategie verwendet `ExperimentRun.environment`, um jedem Run eine einheitliche Richtlinie zuzuordnen. Für Local Intelligence handelt es sich weiterhin um eine Richtlinie und um Bereinigungsbausteine, nicht um eine aktive Aufbewahrungsdurchsetzung für MongoDB-Run-Snapshots.

Verwandte Dokumente: [Architektur- und Ablagekonventionen](#) und [Beispiele zum Datenbankverbindungsentwurf](#).

Local-Intelligence-Datenmodell v0.1

Status: Beschreibender Überblick über das Datenmodell (Version 0.1). Dieses Dokument beschreibt die im Ausgangstext dargestellten Modellkonzepte und Beziehungen. Der Überblick garantiert nicht, dass jede beschriebene Beziehung exakt wie dargestellt implementiert ist.

1. Gesamtübersicht

Das Datenmodell lässt sich vereinfacht in folgende Ebenen einteilen:

1. Experiment-Ebene

- `ExperimentRun`
- `ExperimentParticipant`

2. Agent- und Neighborhood-Ebene

- Agent
- AgentState
- AgentStateHistory
- Neighborhood
- NeighborRelation

3. Signal- und Propagation-Ebene

- Signal
- PropagationEvent
- Rule
- RuleSet
- Threshold

4. Beobachtungs- und Ergebnisebene

- Observation
- ObservationMetric
- CollectiveBehaviorResult

5. Betriebs-, Fehler- und Protokollebene

- ProtocolEvent
- TechnicalWarning
- FailureState

6. Session- und Medienebene

- Session
- Media
- State

2. Zentrale Beziehung

Die wichtigste Kette ist:

```
ExperimentRun
|
├─ ExperimentParticipant
|
├─ Agent
```



3. ExperimentRun

ExperimentRun ist die zentrale Entität für einen konkreten Experimentlauf.

Das Modell enthält unter anderem:

- `sessionId` → Verknüpfung mit einer Session
- `status` → zum Beispiel `created`, `running`, `paused`, `finished`, `failed` oder `cancelled`
- Lifecycle- und Statistikdaten
- `activeRuleSetId` → aktives RuleSet
- Umgebung bzw. Experimentkontext

Viele andere Modelle verwenden `experimentRunId`, um ihre Daten einem bestimmten Experimentlauf zuzuordnen.

4. ExperimentParticipant

ExperimentParticipant verknüpft Teilnehmer mit einem ExperimentRun. Dadurch kann ein Experimentlauf mehrere Teilnehmer umfassen.

Beziehung:

ExperimentRun 1 — N ExperimentParticipant

5. Agent

Agent repräsentiert ein einzelnes Gerät bzw. einen Teilnehmer innerhalb der Crowd-/Simulationslogik.

Wichtige Beziehungen und Attribute:

- `stateId` → `AgentState`
- `neighborhoodId` → `Neighborhood`
- der Agent besitzt Positionsdaten
- `lastSeen` und `status` unterstützen die Online-/Offline-Überwachung
- `deviceId` identifiziert den Agent

In der vorgeschlagenen Architektur gehört ein Agent genau zu **einem** `Neighborhood`.



6. AgentState und AgentStateHistory



Damit lassen sich sowohl der aktuelle Zustand als auch die historische Entwicklung eines Agents nachvollziehen.

7. Neighborhood

`Neighborhood` repräsentiert einen lokalen Bereich innerhalb eines Experiments.

Es enthält unter anderem:

- `experimentRunId`

- name
- agentIds
- configuration
- globale Bounds bzw. räumliche Informationen

Die Position eines Agents kann innerhalb seines Neighborhoods lokal interpretiert werden.

Beispiel: Neighborhood N01

```
1 2 3
4 5 6
7 8 9
```

Ein Agent kann beispielsweise folgende lokale Koordinaten besitzen:

```
{
  "position": {
    "row": 2,
    "col": 3
  }
}
```

8. NeighborRelation

NeighborRelation beschreibt eine direkte Beziehung zwischen zwei Agents. Typischerweise lässt sich die Beziehung so darstellen:

```
Agent A — NeighborRelation — Agent B
```

Im aktuellen Architekturkonzept gehört ein Agent nur zu einem Neighborhood. Beziehungen zwischen Agents können dennoch innerhalb eines Neighborhoods bestehen.

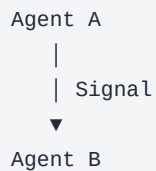
Wird ein ausgefallener Agent ersetzt, werden seine bestehenden Beziehungen auf den Ersatz-Agent übertragen.

9. Signal

Signal repräsentiert eine Nachricht bzw. ein Signal, das von einem Agent ausgeht und möglicherweise an einen anderen Agent adressiert ist.

Wichtige Beziehungen:

- `sourceAgentId` → Quell-Agent
- `targetAgentId` → Ziel-Agent
- `experimentRunId` → Experimentlauf
- `neighborhoodId` → lokaler Bereich
- `correlationId` → Korrelation zusammengehöriger Vorgänge



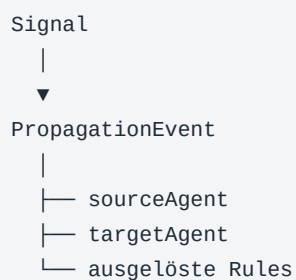
10. PropagationEvent

`PropagationEvent` dokumentiert, was während der Weiterleitung mit einem Signal geschieht.

Insbesondere verknüpft es:

- `signalId`
- `experimentRunId`
- `neighborhoodId`
- `sourceAgentId`
- `targetAgentId`
- `triggeredRuleIds`
- `status`
- `correlationId`

Beziehung:



Signal beschreibt somit die Nachricht; PropagationEvent beschreibt ihre Weiterleitung bzw. Verarbeitung.

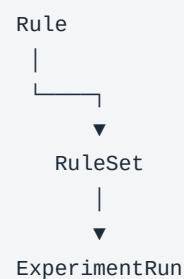
11. Rule, RuleSet und Threshold

Rule beschreibt eine einzelne Regel.

RuleSet fasst mehrere Regeln zu einer Konfiguration zusammen.

Threshold kann Schwellenwerte für Entscheidungs- oder Verhaltenslogik repräsentieren.

Vereinfachte Beziehung:

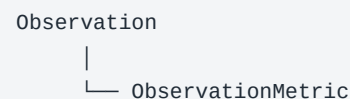


Über activeRuleSetId kann der ExperimentRun auf das aktuell aktive Regelset verweisen.

12. Observation und ObservationMetric

Observation dient zur Speicherung einer beobachteten Situation bzw. eines beobachteten Vorgangs.

ObservationMetric speichert Messwerte bzw. Kennzahlen, die zu einer Beobachtung gehören.



Dadurch können die Rohbeobachtung und die dazu gespeicherten Messwerte getrennt behandelt werden.

13. CollectiveBehaviorResult

CollectiveBehaviorResult speichert die Ergebnisse einer Analyse des kollektiven Verhaltens. Es gehört damit eher zur Ergebnis-/Analytics-Ebene als zur Ebene eines einzelnen technischen Ereignisses.

Agents



Observations / Metrics



CollectiveBehaviorResult

14. ProtocolEvent

ProtocolEvent ist das zentrale Protokoll wichtiger fachlicher und technischer Abläufe.

Beispiele aus den vorhandenen Routes:

- SESSION_CREATED
- SESSION_CONNECTED
- SESSION_DISCONNECTED
- SIGNAL_CREATED
- SIGNAL_STATUS_CHANGED
- PROPAGATION_CREATED
- PROPAGATION_STATUS_CHANGED
- NEIGHBORHOOD_CREATED
- NEIGHBORHOOD_UPDATED
- RULESET_CREATED
- RULESET_ASSIGNED
- AGENT_REPLACED

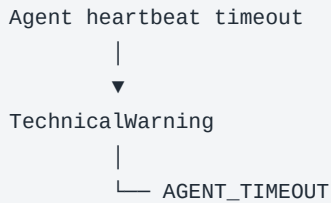
Je nach Ereignis referenziert ein ProtocolEvent beispielsweise:

- experimentRunId
- agentId
- sessionId
- signalId
- propagationEventId

15. TechnicalWarning

TechnicalWarning beschreibt technische Probleme oder Warnungen.

Beispiel:

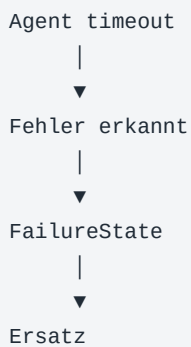


Dieses Modell ist für technische Warnungen vorgesehen und sollte nicht die normale fachliche Ereignisprotokollierung ersetzen.

16. FailureState

FailureState repräsentiert einen tatsächlichen Fehler- bzw. Ausfallzustand.

Beispiel:



Ein TechnicalWarning kann auf ein Problem hinweisen, während FailureState den tatsächlichen Fehlerzustand repräsentiert.

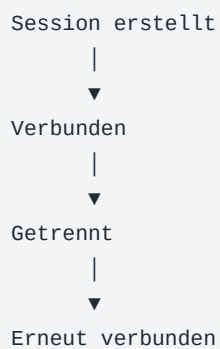
17. Session

Session ist ein vorgeschlagenes Konzept für eine technische Verbindung bzw. Sitzung und kein lokales Modell der aktuellen Local-Intelligence-Laufzeit.

Die umfassendere Crowds-Modellskizze sieht unter anderem folgende Werte vor:

- type
- socketId
- performerId
- isConnected
- lastSeen
- disconnectReason

Lifecycle:

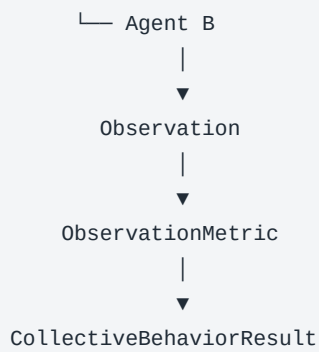


Im vorgeschlagenen Design werden die jeweiligen Zustandsänderungen zusätzlich über `ProtocolEvent` protokolliert; dieser Session-Lifecycle ist in der aktuellen Local-Intelligence-API nicht implementiert.

18. Beispiel eines vollständigen Ablaufs

Ein möglicher Ablauf sieht so aus:





Wichtige Systemereignisse werden parallel protokolliert:



19. Die drei wichtigsten Event-Konzepte

Konzept	Beantwortete Frage
ProtocolEvent	Was ist im System passiert?
TechnicalWarning	Welches technische Problem bzw. Warnsignal wurde erkannt?
FailureState	Welcher tatsächliche Fehler- bzw. Ausfallzustand liegt vor?

Beispiel:



Zusammenfassung

Die Modelle sind miteinander verknüpft und bilden eine zusammenhängende Datenstruktur:

- `ExperimentRun` definiert den Experimentkontext.
- `ExperimentParticipant` verknüpft Teilnehmer mit dem Experiment.
- `Agent` repräsentiert Geräte bzw. Teilnehmer.
- `Neighborhood` organisiert Agents in lokalen Bereichen.
- `NeighborRelation` beschreibt Beziehungen zwischen Agents.
- `AgentState` und `AgentStateHistory` verwalten den aktuellen Zustand und dessen Historie.
- `Signal` beschreibt die Kommunikation.
- `PropagationEvent` beschreibt die Verarbeitung bzw. Weiterleitung eines Signals.
- `Rule`, `RuleSet` und `Threshold` unterstützen die Verhaltenslogik.
- `Observation` und `ObservationMetric` speichern Beobachtungen und Messwerte.
- `CollectiveBehaviorResult` speichert Analyseergebnisse.
- `ProtocolEvent` protokolliert wichtige Systemereignisse.
- `TechnicalWarning` protokolliert technische Warnungen.
- `FailureState` beschreibt tatsächliche Fehlerzustände.
- `Session` verwaltet technische Verbindungen.
- `State` und `Media` unterstützen Medien- und Zustandsdaten.

Die Workspace-Implementierungen dieser Konzepte umfassen [ExperimentRun](#), [ExperimentParticipant](#), [Agent](#), [AgentState](#), [AgentStateHistory](#), [Neighborhood](#), [NeighborRelation](#), [Signal](#), [PropagationEvent](#), [Rule](#), [RuleSet](#), [Threshold](#), [Observation](#), [ObservationMetric](#), [CollectiveBehaviorResult](#), [ProtocolEvent](#), [TechnicalWarning](#), [FailureState](#) und [State](#).

Vorschlag für das Local-Intelligence-Datenmodell v0.1

Status: Vorschlag (Version 0.1). Dieses Dokument stellt ein vorgeschlagenes Datenmodell zur Diskussion und Weiterentwicklung vor. Es behauptet nicht, dass jedes Feld und jede Beziehung bereits im aktuellen Backend implementiert ist.

Agent

Bezug zu Crowds

Das bestehende Crowds-Backend verwaltet bereits verbundene Performer-Geräte, aktive Sessions und Echtzeitkommunikation über Socket.IO.

Ein Agent erweitert diese Laufzeitumgebung, indem er einen Performer als autonomen Teilnehmer eines Local-Intelligence-Experiments repräsentiert. Statt einen neuen Client-Typ einzuführen, bietet Agent eine logische Abstraktion, die einen eigenen Zustand verwalten, Signals austauschen und an lokalen Interaktionen teilnehmen kann.

Felder

Feld	Typ	Beschreibung
id	string	Eindeutige Kennung des Agents
deviceId	string	Kennung des verbundenen Crowds-Geräts
stateId	AgentState	Aktueller Zustand des Agents
neighborhoodId	string	Neighborhood, zu dem der Agent gehört

Beziehungen

- gehört zu genau einem Neighborhood
- besitzt einen aktuellen AgentState
- kann Signals senden und empfangen
- nimmt über sein Neighborhood an einem ExperimentRun teil
- kann mit mehreren NeighborRelations verbunden sein

Beispieldatensatz

Lesbares Beispiel:

```
{
```

```
"id": "agent-17",
"deviceId": "phone-17",
"stateId": "state-19",
"neighborhoodId": "7"
}
```

AgentState

Bezug zu Crowds

Das Crowds-Backend speichert den globalen Anwendungszustand bereits über `State.js`. Dadurch kann das System die Gesamtlaufzeit einer Session synchronisieren.

AgentState führt eine separate Ebene ein, um den internen Zustand eines einzelnen Agents während eines Local-Intelligence-Experiments abzubilden. Während sich das bestehende Backend auf globale Synchronisierung konzentriert, kann jeder Agent mit AgentState sein Verhalten als Reaktion auf lokale Interaktionen und Regeln unabhängig ändern.

Felder

Feld	Typ	Beschreibung
id	string	Eindeutige Kennung
agentId	string	Referenz auf den Agent
state	string (enum)	Aktueller Zustand (inactive, waiting, active, ...)
updatedAt	datetime	Zeitpunkt der letzten Zustandsänderung

Beziehungen

- gehört zu genau einem Agent
- kann von einer oder mehreren Rules aktualisiert werden
- kann während eines ExperimentRun aufgezeichnet werden

Beispieldatensatz

Lesbares Beispiel:

```
{
  "id": "state-19",
  "agentId": "agent-17",
  "state": "active",
  "updatedAt": "2026-06-25T14:23:41Z"
}
```

Neighborhood

Bezug zu Crowds

Das aktuelle Crowds-Backend verwaltet Sessions und verbundene Performer, kennt jedoch keine lokalen Interaktionsgruppen.

Neighborhood führt diese zusätzliche Abstraktion ein, indem es Agents innerhalb eines ExperimentRun gruppiert. Es definiert den lokalen Interaktionskontext, in dem Signals weitergeleitet und Rules ausgewertet werden können. Die Zuordnung zu einem Neighborhood kann aus bestehenden Session-Informationen, Sitzplänen oder experimentspezifischen Konfigurationen abgeleitet werden.

Felder

Feld	Typ	Beschreibung
id	string	Eindeutige Kennung
experimentRunId	string	Referenz auf den ExperimentRun
bounds	Object	Globale Position des Neighborhoods (rowStart, rowEnd, colStart, colEnd), einschließlich der Grenzen und 1-basiert. Wird ausschließlich zur Ableitung der Nachbarschaft verwendet.
agentIds	string []	Referenzen auf die Agents des Neighborhoods. Ein Agent gehört genau zu einem Neighborhood.
neighbors	Object []	Angrenzende Neighborhoods, jeweils dargestellt als (neighborhoodId, direction, distance)

Mögliches zukünftiges Feld

name (string) — Optionale, für Betreiber lesbare Kennung. Dieses Feld ist für das Kernmodell nicht erforderlich, könnte jedoch die Bedienbarkeit bei Einrichtung und Überwachung von Experimenten verbessern.

Beziehungen

- gehört zu genau einem ExperimentRun
- enthält mehrere Agents
- enthält mehrere NeighborRelations
- kann während eines ExperimentRun Signals empfangen

Beispieldatensatz

Lesbares Beispiel:

```
{
  "id": "7",
  "experimentRunId": "12",
  "bounds": { "rowStart": 1, "rowEnd": 3, "colStart": 4, "colEnd": 6 },
  "agentIds": [
    "agent-17",
    "agent-18",
    "agent-19",
    "agent-20"
  ],
  "neighbors": [
    { "neighborhoodId": "6", "direction": "west", "distance": 3 },
    { "neighborhoodId": "8", "direction": "east", "distance": 3 }
  ]
}
```

Signal

Bezug zu Crowds

Das bestehende Crowds-Backend verteilt bereits synchronisierte Ereignisse zwischen Controller, Performern und Besuchern über Socket.IO.

Signal baut auf dieser Kommunikationsinfrastruktur auf und repräsentiert experimentspezifische Informationen, die innerhalb eines Local-Intelligence-Experiments ausgetauscht werden. Signals können von einem ExperimentRun oder einem Agent ausgehen. So lassen sich lokale Interaktionen, Regelauswertung und Signalweiterleitung unabhängig vom zugrunde liegenden Transportmechanismus modellieren.

Felder

Feld	Typ	Beschreibung
id	string	Eindeutige Kennung
type	string	Signaltyp
sourceId	string	Ursprung des Signals (Agent oder ExperimentRun)
targetId	string	Ziel-Agent oder Ziel-Neighborhood
payload	Object	Signalspezifische Daten
timestamp	datetime	Zeitpunkt der Signalerstellung

Beziehungen

- kann von einem ExperimentRun ausgehen
- kann von einem Agent ausgehen
- kann an ein Neighborhood oder einen Agent adressiert sein
- kann eine oder mehrere Rules auslösen
- kann mehrere PropagationEvents erzeugen

Beispieldatensatz

Lesbares Beispiel:

```
{
  "id": "501",
  "type": "pulse",
  "sourceId": "agent-17",
  "targetId": "agent-18",
  "payload": {
    "strength": 1.0,
  }
}
```

```
  "color": "red"
},
"timestamp": "2026-06-25T19:30:00Z"
}
```

RuleSet

Bezug zu Crowds

Ein RuleSet definiert die Verhaltenslogik, die während einer Crowds-Session angewendet werden kann. Im bestehenden Backend gibt es kein explizites Regelsystem; das Echtzeitverhalten wird stattdessen über Socket.IO-Events verarbeitet (controller → performer/visitor).

RuleSets führen eine strukturierte Abstraktionsebene ein, die eingehende Socket.IO-Signale interpretiert und in Verhaltensänderungen auf Agent-Ebene übersetzt, zum Beispiel in AgentState-Aktualisierungen und Signalweiterleitung.

Felder

Feld	Typ	Beschreibung
id	string	Eindeutige Kennung
experimentRunId	string	Referenz auf den ExperimentRun
ruleIds	string[]	Liste der Verhaltensregeln
active	boolean	Gibt an, ob das RuleSet derzeit aktiv ist

Mögliches zukünftiges Feld

name (string) — Optionale, für Betreiber lesbare Kennung. Dieses Feld ist für das Kernmodell nicht erforderlich, könnte jedoch die Bedienbarkeit bei Einrichtung und Überwachung von Experimenten verbessern.

Beziehungen

- gehört zu genau einem ExperimentRun
- referenziert mehrere Rules (über ruleIds)

- wertet Signals aus
- kann AgentState aktualisieren

Beispieldatensatz

Lesbares Beispiel:

```
{
  "id": "ruleset-1",
  "experimentRunId": "12",
  "active": true,
  "ruleIds": [
    "rule-1",
    "rule-2"
  ]
}
```

Rule

Bezug zu Crowds

Rules definieren, wie eingehende Socket.IO-Events (Aktualisierungen von Controller/Performer/Visitor) im Kontext von Local Intelligence interpretiert werden. Im Crowds-Backend ist diese Logik derzeit implizit in Socket-Handletern (`controller.js`, `performer.js`) enthalten. Rules machen sie explizit und konfigurierbar.

Felder

Feld	Typ	Beschreibung
id	string	Eindeutige Kennung
type	string	Regeltyp (z. B. threshold, pattern-match)
signalType	string	Signaltyp, auf den die Rule reagiert
action	string	Auszuführende Aktion (activate, propagate, sync)

Feld	Typ	Beschreibung
threshold	number	Aktivierungsschwelle

Beziehungen

- gehört zu einem RuleSet
- wertet ein Signal aus
- kann AgentState aktualisieren
- kann ein PropagationEvent erzeugen

Beispieldatensatz

Lesbares Beispiel:

```
{
  "id": "rule-1",
  "type": "threshold",
  "signalType": "pulse",
  "action": "activate",
  "threshold": 2
}
```

ExperimentRun

Bezug zu Crowds

Ein ExperimentRun entspricht annähernd einer Crowds-„Session“. Im Backend wird eine Session bereits durch `Session.js` repräsentiert und über `Socket.IO-Namespaces` koordiniert.

ExperimentRun erweitert dieses Konzept um:

- Modellierung auf Agent-Ebene
- Neighborhood-Struktur
- Regelausführung
- eine Observability-Ebene

Felder

Feld	Typ	Beschreibung
id	string	Eindeutige Kennung
sessionId	string	Referenz auf eine Crowds-Session
status	string	running/paused/finished
startedAt	datetime	Startzeitpunkt
endedAt	datetime	Endzeitpunkt

Beziehungen

- verwendet eine Crowds-Session (`Session.js`)
- enthält mehrere Neighborhoods
- enthält ein RuleSet
- erzeugt ObservationMetrics
- erzeugt CollectiveBehaviorResults

Beispieldatensatz

Lesbares Beispiel:

```
{
  "id": "exp-1",
  "sessionId": "1",
  "status": "running",
  "startedAt": "2026-06-25T19:00:00Z",
  "endedAt": null
}
```

PropagationEvent

Bezug zu Crowds

PropagationEvents repräsentieren Vorgänge, die bei Socket.IO-Broadcasts derzeit implizit ablaufen:

```
Controller sendet Event → Backend leitet es weiter → Performer empfängt es
```

Im Local-Intelligence-Kontext wird jeder Broadcast zu einem nachvollziehbaren Weiterleitungsschritt. Das ist besonders nützlich, weil das Backend bereits Echtzeit-Eventflüsse über folgende Endpunkte bereitstellt:

- /controller
- /performer
- /visitor

Felder

Feld	Typ	Beschreibung
id	string	Eindeutige Kennung
signalId	string	Zugehöriges Signal
sourceId	string	Quelle des Signals
targetId	string	Ziel des Signals
timestamp	datetime	Zeitpunkt der Weiterleitung
status	string	success/blocked/delayed

Beziehungen

- gehört zu einem Signal
- verbindet zwei Agents
- kann eine Regelauswertung auslösen
- ist Teil der Nachverfolgung eines ExperimentRun

Beispieldatensatz

Lesbares Beispiel:

```
{
  "id": "prop-88",
  "signalId": "501",
  "sourceId": "agent-17",
  "targetId": "agent-18",
  "timestamp": "2026-06-25T19:30:01Z",
  "status": "success"
}
```

ObservationMetric

Bezug zu Crowds

Das Crowds-Backend stellt `/metrics` bereits bereit (In-Memory-Zähler für Verbindungen und Nachrichten). ObservationMetrics erweitern dies um Analysen auf Experiment-Ebene. Statt ausschließlich den Systemzustand zu erfassen, misst das Modell nun auch Muster kollektiven Verhaltens.

Felder

Feld	Typ	Beschreibung
id	string	Eindeutige Kennung
experimentRunId	string	Referenz auf den ExperimentRun
type	string	Metriktyp (sync, latency, clustering)
value	number	Gemessener Wert
timestamp	datetime	Messzeitpunkt

Beziehungen

- gehört zu einem ExperimentRun

- wird aus PropagationEvents berechnet
- wird aus AgentState-Änderungen abgeleitet
- wird zur Auswertung von CollectiveBehaviorResult verwendet

Beispieldatensatz

Lesbares Beispiel:

```
{
  "id": "metric-1",
  "experimentRunId": "exp-1",
  "type": "synchronization-score",
  "value": 0.87,
  "timestamp": "2026-06-25T19:35:00Z"
}
```

CollectiveBehaviorResult

Bezug zu Crowds

CollectiveBehaviorResult repräsentiert das interpretierte Ergebnis eines Local-Intelligence-ExperimentRun. Während sich das Crowds-Backend derzeit über `/metrics` auf Echtzeitsynchronisierung und Systemzustand konzentriert, führt diese Entität eine semantische Interpretationsebene für emergentes Gruppenverhalten ein.

Das Ergebnis wird berechnet aus:

- ObservationMetrics (quantitative Signale wie Synchronisierung, Latenz und Clustering)
- PropagationEvents (Weiterleitungswege der Signals durch das System)
- AgentState-Übergängen (Reaktionen der Agents im Zeitverlauf)

In der bestehenden Crowds-Architektur würde dieses Ergebnis derzeit nicht direkt gespeichert. Es könnte jedoch nach einer Session aus MongoDB-Session-Logs und Socket.IO-Event-Traces abgeleitet werden.

Felder

Feld	Typ	Beschreibung
id	string	Eindeutige Kennung

Feld	Typ	Beschreibung
experimentRunId	string	Referenz auf den ExperimentRun
type	string	Typ des emergenten Verhaltens (sync, clustering, wave, consensus)
score	number	Normalisierte Stärke des beobachteten Verhaltens (0–1)
confidence	number	Messsicherheit auf Grundlage der Datenvollständigkeit
startTime	datetime	Startzeitpunkt
endTime	datetime	Endzeitpunkt
summary	string	Für Menschen lesbare Interpretation des Ergebnisses

Beziehungen

- gehört zu genau einem ExperimentRun
- wird aus ObservationMetrics abgeleitet
- wird aus PropagationEvents abgeleitet
- wird aus AgentState-Änderungen abgeleitet
- fasst das Systemverhalten innerhalb eines Neighborhoods oder des gesamten ExperimentRun zusammen

Beispieldatensatz

Lesbares Beispiel:

```
{
  "id": "cbr-1",
  "experimentRunId": "exp-1",
  "type": "synchronization",
  "score": 0.91,
  "confidence": 0.84,
  "startTime": "2026-06-25T19:20:00Z",
```

```
"endTime": "2026-06-25T19:35:00Z",  
"summary": "Nach wiederholter Weiterleitung von Pulse-Signalen durch das Netzwerk entstand eine  
hohe Synchronisierung über alle Agents hinweg."  
}
```

Zuordnung: bestehendes Backend → Local Intelligence

Crowds-Backend	Local-Intelligence-Konzept
Session.js	ExperimentRun
State.js	AgentState
Socket.IO-Events	Signal
Controller-/Performer-/Visitor-Sockets	Agent-Interaktionen
/metrics-Endpunkt	ObservationMetric (System-Basiswerte)
MongoDB-Session-Speicherung	Persistenz von ExperimentRun + Neighborhood
Echtzeit-Broadcasts	PropagationEvent

Die Bezeichner und Feldnamen oben bleiben wie im Vorschlag v0.1 erhalten. Die Implementierung kann davon abweichen; siehe die entsprechenden Workspace-Modelle: [Agent](#), [AgentState](#), [Neighborhood](#), [Signal](#), [Rule](#), [RuleSet](#), [ExperimentRun](#), [PropagationEvent](#), [ObservationMetric](#) und [CollectiveBehaviorResult](#).

Entitätenkarte von Local Intelligence (v0.1)

Dies ist eine übergeordnete konzeptionelle Karte der Domäne des Prototyps. Die Beziehungen erläutern die fachlichen Zusammenhänge; daraus folgt nicht, dass jede Verknüpfung als Fremdschlüssel erzwungen oder in einer eigenen Datenbank-Collection gespeichert wird.

Überblick über die Entitäten

- **Experiment:** ExperimentRun bildet den Kontext eines Laufs; ExperimentParticipant ordnet ihm Teilnehmende zu.
- **Agenten und Nachbarschaften:** Agent, AgentState, AgentStateHistory, Neighborhood und NeighborhoodRelation beschreiben Teilnehmende, ihren aktuellen und historischen Zustand sowie lokale Verbindungen. NeighborhoodRelation ist der Modellname in der Implementierung; in den Ausgangsnotizen steht auch NeighborRelation.
- **Signale und Regeln:** Signal und PropagationEvent beschreiben Nachrichten und deren Weiterleitung; RuleSet, Rule und Threshold beschreiben die Verhaltenskonfiguration.
- **Beobachtungen und Ergebnisse:** Observation, ObservationMetric und CollectiveBehaviorResult erfassen Beobachtungen, Messwerte und Analyseergebnisse.
- **Betrieb und Diagnose:** ProtocolEvent, TechnicalWarning und FailureState stehen für Ereignisprotokolle, technische Warnungen und Fehlerzustände.
- **Unterstützendes Modell:** State definiert im Prototyp verwendete Zustandsdaten.

Konzeptionelle Beziehungen

```
ExperimentRun
├─ ExperimentParticipant
├─ Agent — AgentState / AgentStateHistory
│   └─ Neighborhood — NeighborhoodRelation — Agent
│       └─ Signal — PropagationEvent
├─ RuleSet — Rule — Threshold
├─ Observation — ObservationMetric
├─ CollectiveBehaviorResult
├─ ProtocolEvent
├─ TechnicalWarning
└─ FailureState
```

Der Zustand eines Agenten und seine Zugehörigkeit zu einer Nachbarschaft werden getrennt modelliert. Signale können von einem Agenten oder von einer experimentweiten Steuerungslogik erzeugt werden; Weiterleitung und Regelauswertung hängen von der Experimentkonfiguration ab.

Implementierungsstatus und offene Fragen

Das Repository exportiert aus dem [Model-Index](#) Model-Schemas für Agent, AgentState, AgentStateHistory, CollectiveBehaviorResult, ExperimentParticipant, ExperimentRun, FailureState, Neighborhood, NeighborhoodRelation, Observation, ObservationMetric, PropagationEvent, ProtocolEvent, Rule, RuleSet, Signal, State, TechnicalWarning und

Threshold. Damit ist das Vorhandensein der Model-Definitionen belegt; es bedeutet nicht automatisch, dass jeder Ablauf oder jede Beziehung vollständig implementiert oder über diese einzelnen Modelle persistiert wird. Die Persistenz von Experimentläufen speichert außerdem einen Snapshot des Laufs.

Session und Media kommen in der umfassenderen Modellskizze vor, besitzen im aktuellen Model-Index jedoch keine entsprechenden exportierten Schemas. Sie sind daher als vorgeschlagene oder externe Konzepte zu behandeln, nicht als implementierte lokale Modelle. Insbesondere ist `ExperimentRun.sessionId` eine optionale Kennung für eine externe Session und kein Verweis auf ein lokales Session-Modell.

Die ursprüngliche Karte stellt die Frage, ob die Kommunikation auf Experimentebene und zwischen Agenten durch eine gemeinsame Entität `Signal` abgebildet werden soll. Diese Modellierungsfrage bleibt offen; die Karte entscheidet sie nicht.

Beispiele für Signalflüsse

```
Agent
↓
Signal
↓
NeighborhoodRelation
↓
Agent
↓
Rule
↓
AgentState

ExperimentRun
↓
Signal
↓
Neighborhood
↓
Agent
↓
Signal
↓
NeighborhoodRelation
↓
Agent
↓
Rule
↓
AgentState
```

Siehe auch [Prototyp: Datenbank und Datenmodell](#) und [Beispiel: Signal, Autonomie und Folgeaktion](#).

API-Routen

Dieses Dokument enthält eine vorgeschlagene Routenübersicht und das jeweils zugehörige Verhalten. Es handelt sich um eine Spezifikation und nicht um eine Bestätigung, dass diese Endpunkte oder Funktionen in der aktuellen Codebasis implementiert sind. Vor der Annahme, dass eine Route verfügbar ist, muss sie anhand der laufenden Anwendung und des Quellcodes überprüft werden.

Hinweis zum aktuellen Implementierungsstand: Die Routen dieses Vorschlags sind in der aktuellen Local-Intelligence-Anwendung nicht eingebunden, sofern dies nicht separat anhand des Quellcodes verifiziert wurde. Die nachfolgend beschriebenen Session-, Media-, Simulator-, Analytics- und Recovery-Routen sind Entwürfe; ihre MongoDB-Persistenz ist nicht durch den aktuellen Run-Snapshot-Persistenzpfad implementiert.

Die Endpunktzeichenfolgen sind exakt wie in der Vorlage übernommen. Insbesondere enthalten `PATCH//experiments/:id/complete` und `GET//experiments/:id/export` dort doppelte Schrägstriche. Ob diese beabsichtigt sind, ist unklar und sollte überprüft werden.

Experimente

- `POST /experiments` — Erstellt einen Experimentlauf, initialisiert dessen Lebenszyklus, verknüpft ihn mit einer Session und setzt den Experimentstatus auf `running`.
- `PATCH//experiments/:id/complete` — Schließt einen Experimentlauf ab, speichert die Endzeit, aktualisiert den endgültigen Status, die Zusammenfassung und das Ergebnis und protokolliert ein `ProtocolEvent`.
- `GET /experiments/:id/summary` — Gibt eine statistische Zusammenfassung eines Experiments zurück, einschließlich Participants, Signals, PropagationEvents, Observations, Warnungen, Fehlerzuständen und Ergebnissen zum kollektiven Verhalten.
- `GET//experiments/:id/export` — Exportiert die experimentbezogenen Daten, einschließlich Agents, Participants, Signals, PropagationEvents, Observations, ProtocolEvents, Warnungen, Fehlerzuständen und Analyseergebnissen.
- `GET /experiments/:id/timeline` — Gibt die chronologische Abfolge der während eines Experiments gespeicherten ProtocolEvents zurück.

Agents und Neighborhoods

- `POST /agents` — Registriert ein mobiles Gerät als Agent, erstellt dessen Anfangszustand und setzt den Agent auf online.

- `PATCH /agents/:id/state` — Aktualisiert den aktuellen Zustand eines Agents, speichert den Zustandsübergang in der Historie und protokolliert die Änderung als `ProtocolEvent`.
- `POST /neighborhoods` — Erstellt eine `Neighborhood` für ein Experiment und ordnet ihr die ausgewählten Agents zu.

Signals und Propagation

- `POST /signals` — Erstellt ein von einem Agent erzeugtes Signal, weist ihm eine `Correlation ID` zu und protokolliert die Erstellung.
- `PATCH /signals/:id/status` — Aktualisiert den Verarbeitungsstatus eines Signals und protokolliert die Statusänderung.
- `POST /propagation` — Erstellt ein `PropagationEvent`, das die Übertragung eines Signals zwischen zwei Agents darstellt, und protokolliert dieses Ereignis.
- `PATCH /propagation/:id/status` — Aktualisiert den Status eines `PropagationEvents` (zum Beispiel `sent`, `received`, `processed` oder `failed`) und protokolliert den Übergang.

Diagnose und Authentifizierung

- `GET /diagnostics` — Gibt einen Überblick über den aktuellen Systemzustand zurück, einschließlich Agent-Status, laufender Experimente und Aktivitäten der letzten 30 Minuten.
- `GET /experiments/:id/diagnostics` — Gibt detaillierte Diagnoseinformationen für ein bestimmtes Experiment zurück, einschließlich Statistiken, Warnungen, Fehlerzuständen, aktuellen `ProtocolEvents` und Ergebnissen der Verhaltensanalyse.
- `POST /auth` — Authentifiziert den Controller mit einem Passwort, begrenzt fehlgeschlagene Anmeldeversuche und gibt ein 24 Stunden gültiges JWT zurück.

Analytics

- `GET /analytics/events` — Gibt die neuesten dauerhaft gespeicherten `ProtocolEvents` aus der Datenbank zurück. Mit dem optionalen Parameter `limit` lässt sich die Anzahl der zurückgegebenen Ereignisse steuern.
- `GET /analytics/summary` — Gibt eine Analytics-Zusammenfassung für einen angegebenen Zeitraum zurück, einschließlich der Gesamtzahl der `ProtocolEvents` und ihrer Verteilung nach Ereignistyp und Namespace.

RuleSets

- `POST /rulesets` — Erstellt ein RuleSet für ein vorhandenes Experiment, speichert dessen Regeln und Konfiguration und protokolliert die Erstellung als ProtocolEvent.
- `PATCH /rulesets/:id/assign` — Weist einem Experiment ein vorhandenes RuleSet zu, setzt es als aktives RuleSet, aktiviert es und protokolliert die Zuweisung als ProtocolEvent.

Sessions

- `POST /sessions` — Erstellt eine Session für einen verbundenen Client oder Performer, speichert Socket- und Performer-Informationen und protokolliert die Erstellung als ProtocolEvent.
- `GET /sessions/:id` — Gibt eine Session anhand ihrer ID zurück, einschließlich Verbindungsstatus und zugehöriger Informationen.
- `PATCH /sessions/:id/disconnect` — Trennt eine Session, speichert den Trennungsgrund und den Zeitpunkt der letzten Aktivität und protokolliert die Trennung als ProtocolEvent.
- `PATCH /sessions/:id/connect` — Verbindet eine Session erneut, löscht den vorherigen Trennungsgrund, aktualisiert den Zeitpunkt der letzten Aktivität und protokolliert die erneute Verbindung als ProtocolEvent.

Simulator

- `GET /simulator/scenarios` — Gibt die verfügbaren Simulator-Szenarien zurück, die für einen Lauf ausgewählt werden können.
- `GET /simulator/runs` — Gibt eine Liste der erstellten oder ausgeführten Simulator-Runs zurück.
- `DELETE /simulator/reports` — Löscht bzw. bereinigt erstellte Simulator-Reports und gibt das Ergebnis der Bereinigung zurück.
- `POST /simulator/runs` — Startet einen Simulator-Run für ein ausgewähltes Szenario und gibt den neu erstellten Run zurück.
- `GET /simulator/runs/:runId` — Gibt die aktuellen Informationen und den Status eines bestimmten Simulator-Runs zurück.
- `GET /simulator/runs/:runId/report` — Gibt den Report eines abgeschlossenen Simulator-Runs zurück, indem der Markdown-Report gelesen und als HTML-Dokument dargestellt wird.

Medien

- `GET /visuals` — Gibt die verfügbaren visuellen Mediendateien und geparsten Metadaten zurück. Wenn möglich, werden Daten aus MongoDB verwendet; andernfalls werden Dateien aus dem Verzeichnis `public/visuals` gelesen.
- `GET /sounds` — Gibt die verfügbaren Audiodateien und geparsten Metadaten zurück. Wenn möglich, werden Daten aus MongoDB verwendet; andernfalls werden Dateien aus dem Verzeichnis `public/sounds` gelesen.
- `GET /mp3s` — Gibt die verfügbaren MP3-Dateien und geparsten Metadaten zurück. Wenn möglich, werden Daten aus MongoDB verwendet; andernfalls werden Dateien aus dem Verzeichnis `public/mp3s` gelesen.

Recovery

- `GET /recovery/database` — Gibt die aktuelle MongoDB-Auslastung zurück, einschließlich Datenbankstatistiken, Anzahl der Collections, Speicherbelegung und Dokumentanzahlen für Sessions, States und Medien.
- `DELETE /recovery/database/sessions` — Löscht getrennte Sessions aus der Datenbank und gibt die Anzahl der gelöschten Sessions sowie die aktualisierte Datenbankauslastung zurück.

Weiterführende Dokumentation

- [Überblick und API-Routen](#)
- [Ereignistaxonomie und Schema-Governance](#)

Übergabe zur Data Layer: Datenmodelle und API-Routen

Dieses Dokument fasst konzeptionelle Datenmodelle und vorgeschlagene Routen für die Crowds Local Intelligence / Data Layer zusammen. Die nachfolgend dargestellten Modellbeziehungen und das Routenverhalten entsprechen der dokumentierten Architektur und dem vorgesehenen Entwurf. Sie belegen nicht, dass jedes Modell, jede Route oder jede Funktion in der aktuellen Codebasis vorhanden ist. Die Implementierung muss separat überprüft werden, bevor ein Eintrag als verfügbar vorausgesetzt wird. Die ausführliche Routenspezifikation steht in der [Routenübersicht](#).

1. Datenmodelle

Der Entwurf der Crowds Local Intelligence / Data Layer beschreibt Datenmodelle für Experimente, Agents, Neighborhoods, Signale, Propagation, Beobachtungen und technische Ereignisse.

Zentrale Modelle

- **ExperimentRun** — Repräsentiert einen einzelnen Experimentlauf. Dazu gehören unter anderem Status, Umgebung und Laufzeitinformationen. Das Modell dient als übergeordnete Verbindung zu vielen weiteren Datensätzen.
- **ExperimentParticipant** — Verknüpft Agents bzw. andere Teilnehmende mit einem ExperimentRun.
- **Agent** — Repräsentiert ein Gerät oder einen Teilnehmenden im System. Ein Agent kann unter anderem Status, Position und eine neighborhoodId besitzen.
- **AgentState / AgentStateHistory** — Speichern den aktuellen Zustand eines Agents sowie dessen historische Zustandsänderungen.
- **Neighborhood** — Beschreibt einen lokalen Bereich innerhalb eines Experiments und kann zugehörige Agents enthalten. Im vorgesehenen Architekturmodell wird ein Agent einem Neighborhood zugeordnet; im aktuellen Schema ist diese Zuordnung optional.
- **NeighborRelation** — Beschreibt Beziehungen zwischen Agents innerhalb oder zwischen lokalen Bereichen. Der Bezeichner ist wie im Quelldokument angegeben übernommen.
- **Signal** — Speichert ein Signal, das von einem Agent erzeugt und gegebenenfalls an einen anderen Agent weitergegeben wird. Gespeichert werden unter anderem Quelle, Ziel, Status und correlationId.
- **PropagationEvent** — Dokumentiert die Weiterleitung bzw. Verarbeitung eines Signals zwischen Agents und macht so dessen Ablauf nachvollziehbar.
- **Observation / ObservationMetric** — Speichern Beobachtungen und die zugehörigen Messwerte während eines Experiments.
- **Rule / RuleSet / Threshold** — Bilden Regeln und Konfigurationen ab, die für die Verarbeitung und das Verhalten innerhalb eines Experiments verwendet werden.
- **ProtocolEvent** — Dient der zentralen Ereignisprotokollierung. Wichtige Aktionen, etwa das Erstellen eines Signals, eine Änderung seines Status oder das Erstellen einer Neighborhood, sollen als Ereignisse dokumentiert werden.
- **TechnicalWarning / FailureState** — Dienen der Dokumentation technischer Probleme, Warnungen und Fehlerzustände, beispielsweise eines Agent-Timeouts oder eines Ausfalls.
- **Session** — Ein vorgeschlagenes oder externes Konzept für eine technische Verbindung bzw. Sitzung. Der aktuelle Local-Intelligence-Model-Index exportiert kein lokales Session-Modell.

Beziehungen zwischen den Modellen

Der zentrale Zusammenhang lässt sich vereinfacht so darstellen:

ExperimentRun → Neighborhood → Agent

Die Kommunikation kann wie folgt dargestellt werden:

Agent → Signal → PropagationEvent → Agent

Wichtige Vorgänge werden zusätzlich über ProtocolEvent dokumentiert. Technische Probleme werden über TechnicalWarning oder FailureState erfasst.

Diese Beziehungen beschreiben eine nachvollziehbare Datenstruktur zur Speicherung des Systemzustands, der Kommunikation und der Ereignisse eines Experiments. Sie beschreiben das vorgesehene Datenmodell und bestätigen nicht, dass die aktuelle Implementierung alle Beziehungen durchsetzt.

2. API-Routen

Die nachfolgend aufgeführten Routen beschreiben die vorgesehene Schnittstelle zwischen Frontend bzw. Simulator und Datenbank. Sie decken das Erstellen, Aktualisieren und Abrufen sowie teilweise das Löschen von Daten ab. Dieser Abschnitt ist als Routen- und Verhaltensspezifikation zu verstehen, nicht als verifizierte Liste implementierter Endpunkte.

Experiment- und Lifecycle-Routen

Die Experiment-Routen sollen Experimentläufe und deren Lebenszyklus verwalten.

Als Cleanup-Route ist dokumentiert:

POST /lifecycle/cleanup/test-experiments

Sie soll alte Test-Experimente und zugehörige Daten bereinigen, darunter Participants, Signals, PropagationEvents, Observations und technische Ereignisse. Ob sie aktuell implementiert ist, wird hier nicht bestätigt.

Neighborhood-Routen

- POST /neighborhoods — Erstellt eine Neighborhood und kann Agents direkt zuordnen.
- PATCH /neighborhoods/:id — Aktualisiert beispielsweise Name, Konfiguration oder Agent-Zuordnung.

Das beschriebene Verhalten prüft, ob Agents existieren und ob sie bereits einer anderen Neighborhood zugeordnet sind.

Signal-Routen

- `POST /signals` — Erstellt ein Signal und überprüft unter anderem, ob der Quell-Agent und ein optionaler Ziel-Agent existieren.
- `PATCH /signals/:id/status` — Ändert den Status eines Signals, beispielsweise:

`created → sent → received → processed`

Die Statusänderung soll zusätzlich als `ProtocolEvent` dokumentiert werden.

Propagation-Routen

- `POST /propagation` — Erstellt ein `PropagationEvent` für die Weiterleitung eines Signals.
- `PATCH /propagation/:id/status` — Aktualisiert den Verarbeitungsstatus der Propagation und kann beispielsweise Zeitpunkte für `received` und `processed` speichern.

Auch diese Änderungen sollen über `ProtocolEvent` protokolliert werden.

Session-Routen

- `POST /sessions` — Erstellt eine Session.
- `GET /sessions/:id` — Ruft eine bestimmte Session ab.
- `PATCH /sessions/:id/connect` — Setzt eine Session auf verbunden.
- `PATCH /sessions/:id/disconnect` — Setzt eine Session auf getrennt und speichert den Grund.

Die wesentlichen Änderungen im Session-Lebenszyklus sollen ebenfalls als `ProtocolEvent` gespeichert werden.

RuleSet-Routen

- `POST /rulesets` — Erstellt ein `RuleSet` für ein Experiment.
- `PATCH /rulesets/:id/assign` — Ordnet ein `RuleSet` einem Experiment zu und setzt es als aktives `RuleSet`.

Media-Routen

Die Media-Routen sollen dem System verfügbare Medien bereitstellen:

- GET /visuals
- GET /sounds
- GET /mp3s

Der beschriebene Entwurf bevorzugt Daten aus MongoDB. Wenn dort keine Liste verfügbar ist, wird auf Dateien im Verzeichnis `public` zurückgegriffen. Eine Implementierung dieses Verhaltens ist hiermit nicht bestätigt.

Recovery- und Simulator-Funktionen

Zu den beschriebenen Recovery-Funktionen gehören die Überwachung der MongoDB-Auslastung und das Löschen getrennter Sessions.

Die beschriebenen Simulator-Funktionen umfassen:

- Verfügbare Testszenarien auflisten.
- Simulator-Runs starten.
- Laufende Runs abrufen.
- Reports anzeigen.
- Alte Simulator-Reports löschen.

Das Quelldokument nennt für diese Recovery- und Simulator-Funktionen keine Routenpfade. Daher werden hier keine ergänzt.

Zusammenfassung

Der dokumentierte API-Entwurf deckt folgenden Lebenszyklus ab:

Create → Update → Read → Lifecycle → Logging → Cleanup

Ein wichtiges Entwurfsprinzip ist, dass relevante Systemaktionen nicht nur die zugrunde liegenden Daten verändern, sondern zusätzlich über `ProtocolEvent` nachvollziehbar sein sollen. Dies soll spätere Analysen, Monitoring, Debugging und KPI-Auswertungen unterstützen. Daraus folgt nicht, dass diese Funktionen derzeit implementiert sind.

3. Priorität 8 — Interactive / Taiwan Observability Support

Laut der ursprünglichen Übergabenotiz wurde nur „Rehearsal/test-run grouping“ abgeschlossen. Die übrigen Punkte sind noch nicht fertig und müssen überarbeitet werden. Diese Statusangabe wurde nicht unabhängig überprüft.

Weiterführende Dokumentation

- [Ereignistaxonomie und Schema-Governance](#)
- [API-Routen](#)

Ereignistaxonomie und Schema-Governance v0.1

Dieses Dokument definiert eine einheitliche Ereignistaxonomie und eine Strategie zur Schema-Governance für das Crowds-Ökosystem.

Namenskonvention für Ereignisse

Ereignisnamen verwenden das Format DOMAIN_ACTION: Großbuchstaben, durch Unterstriche getrennt, wobei das Verb am Ende steht.

Beispiele:

- SIGNAL_CREATED
- SIGNAL_STATUS_CHANGED
- PROPAGATION_CREATED
- AGENT_STATE_CHANGED
- EXPERIMENT_COMPLETED

Governance-Regeln

Alle neuen Ereignisse in diesem Projekt müssen folgende Regeln erfüllen:

1. **Das standardisierte Namensformat verwenden.** Ereignisnamen müssen dem Format `DOMAIN_ACTION` entsprechen.
2. **Jedem Ereignis genau eine Kategorie zuweisen.** Zulässige Kategorien sind:
 - `runtime`
 - `protocol`
 - `experiment`
 - `analytics`
3. **Jedes `ProtocolEvent`-Dokument muss die folgenden Pflichtfelder enthalten:**
 - `type`
 - `category`
 - `schemaVersion`
 - `timestamp`
 - `experimentRunId`
 - `agentId`
 - `signalId`
 - `propagationEventId`
 - `correlationId`
 - `detail`
4. **`schemaVersion` erhöhen**, wenn sich die Ereignisnutzdaten inkompatibel ändern.
5. **Vor dem Hinzufügen eines Ereignisses:**
 - Alle referenzierten IDs validieren.
 - Standardisierte Ereignisnamen verwenden.
 - Die Dokumentation aktualisieren, wenn ein neuer Ereignistyp hinzukommt.

Diese Spezifikation soll die Kompatibilität zwischen Crowds, Grid, Interactive, Local Intelligence und zukünftigen Forschungssystemen gewährleisten. Sie beschreibt Governance-Anforderungen und bestätigt nicht, dass jedes Ereignis oder Feld in der aktuellen Codebasis implementiert ist.

Weiterführende Dokumentation

- [Überblick und API-Routen](#)
- [API-Routen](#)

Tickbasierte Simulation

Motivation

Die aktuelle Laufzeit verarbeitet Signals und kann während eines aktiven Experimentlaufs außerdem geplante Simulationsticks ausführen. Der vorhandene Tick-Mechanismus ist jedoch noch nicht die vollständig konfigurierbare Schleife pro Agent, die im folgenden Vorschlag beschrieben wird.

Eine Simulations-Engine, die alle Agenten regelmäßig aktualisiert, könnte kontinuierliche Anpassung und realistischeres kollektives Verhalten ermöglichen. Dies ist eine vorgeschlagene zukünftige Erweiterung und noch kein Bestandteil des aktuellen Laufzeitbetriebs.

Aktueller Ablauf

Signal empfangen

↓

`processSignal()`

↓

Regelauswertung

↓

Schwarmverhalten

↓

Protokollierung

Vorgeschlagener Ablauf

Experimentlauf startet

↓

Simulationsschleife

↓

Für jeden Simulationsttick

↓

Für jeden Agenten

↓

Nachbarschaft analysieren

↓

Regeln auswerten

↓

Schwarmverhalten ausführen

↓

Bei Bedarf Signale erzeugen

↓

Protokollierung

↓

Nächster Tick

Mögliche Vorteile

- Kontinuierliche Anpassung
- Realistischeres Schwarmverhalten
- Zeit für die Entwicklung emergenten Verhaltens
- Selbstorganisation ohne externe Ereignisse
- Grundlage für die Implementierung zukünftiger Algorithmen

Signalverarbeitung

Im vorgeschlagenen Design wären Signale nicht mehr der primäre Auslöser des Systems. Stattdessen würden sie als Ereignisse während eines Simulationsticks verarbeitet.

Damit werden folgende Aspekte voneinander getrennt:

- Zeitsteuerung der Simulation
- Regelauswertung
- Signalweiterleitung

Mögliche Konfiguration

```
{  
  "tickRate": 100,  
  "maxTicks": 10000,  
  "autoStop": true  
}
```

Mögliche zukünftige Erweiterungen

- Variable Tick-Rate
- Simulation pausieren und fortsetzen
- Verteilte Simulation
- Parallele Verarbeitung von Nachbarschaften
- Prioritätsplanung
- Dynamische Aktualisierung von Agenten

Status und Hinweise

Dieses Dokument schlägt eine Erweiterung des vorhandenen Tick-Mechanismus vor. Der aktuelle Scheduler löst standardmäßig alle 1.000 ms einen Simulationstick aus, solange ein Lauf aktiv ist. Die darüber hinaus beschriebenen Tick-Raten, Verarbeitung pro Agent und Planungsoptionen sind Vorschläge und nicht vollständig im aktuellen Laufzeitverhalten enthalten. Erweiterungen sollten das bestehende Verhalten erhalten und anhand des Datenmodells sowie des Persistenzdesigns bewertet werden.

Vorschlag: Intelligentes Signalarouting für die Local Intelligence Layer

Status: Vorschlag. Dieses Dokument beschreibt eine mögliche zukünftige Funktion und behauptet nicht, dass intelligentes Routing bereits implementiert ist.

Motivation

Die aktuelle Architektur bietet bereits eine solide Grundlage:

- Ein Agent kann höchstens einem Neighborhood zugeordnet sein; laut aktuellem Schema ist die Zuordnung optional.
- Neighborhoods bilden durch explizite Verbindungen zwischen Neighborhoods einen Graphen.
- Signals können zwischen verbundenen Neighborhoods weitergeleitet werden.

Der nächste logische Schritt besteht darin, die Signalweiterleitung **intelligent** zu gestalten. Anstatt ein Signal einfach an jeden erreichbaren Neighbor weiterzuleiten, könnte es selbst eine effiziente Route zu seinem Ziel auswählen. Dadurch würde aus einfacher Flutung ein zielgerichtetes, verteiltes Routing.

Aktuelle Situation

Die aktuelle Propagation-Engine leitet ein Signal von einem Agent an benachbarte Agents weiter. Erreicht ein Signal die Grenze eines Neighborhoods, kann es in eines der verbundenen Neighborhoods weitergeleitet werden.

Konzeptionell:



Dieser Ansatz funktioniert für lokale Weiterleitung gut, kann mit zunehmender Größe eines Experiments jedoch ineffizient werden.

Das Problem

Angenommen, ein Experiment besteht aus Dutzenden oder Hunderten von Neighborhoods. Ein Signal startet in **Neighborhood 1** und soll **Neighborhood 10** erreichen.

Ohne Routing-Intelligenz kennt das Signal sein Ziel nicht und bewegt sich Schritt für Schritt durch den Graphen. Mögliche Folgen sind:

- unnötige Weiterleitungen
- doppelte Arbeit
- zusätzliche Latenz
- erhöhter CPU-Verbrauch
- unnötiger Speicherverbrauch
- eine übermäßige Anzahl von Propagation-Events

Mit wachsendem Experiment wird dieses Vorgehen zunehmend ineffizient.

Routingziele

Ein Signal sollte sein Ziel angeben können. Zwei Zieltypen sind sinnvoll.

Ziel-Neighborhood

```
{  
  "targetNeighborhoodId": "NH10"  
}
```

Ziel ist es, das Signal an ein bestimmtes Neighborhood zu liefern.

Ziel-Agent

```
{  
  "targetAgentId": "agent-275"  
}
```

Zunächst ermittelt das System, zu welchem Neighborhood der Agent aktuell gehört:

```
Agent  
  ↓  
Neighborhood
```

↓
Routing

Ist ein Agent einem Neighborhood zugeordnet, lässt sich dieses über die Zuordnung ermitteln.

Warum die Agent-Abfrage nicht aufwendig ist

Auf den ersten Blick könnte es so wirken, als sei die Suche nach einem Agent vor dem Routing ein zusätzlicher, teurer Schritt. In der Praxis ist dieser Aufwand meist vernachlässigbar:

Agent-ID
↓
Neighborhood ermitteln
↓
Route berechnen

Mit einer Hash-Map hat die Suche nach einer Agent-ID typischerweise durchschnittlich eine Laufzeit von $O(1)$. Die Kosten der Routenberechnung hängen von der Graphgröße und dem ausgewählten Algorithmus ab; sie sind nicht zwangsläufig höher als jede Agentensuche. Bei einem Agent-Ziel kommt die Ermittlung des zugehörigen Neighborhoods hinzu.

Intelligentes Routing

Anstatt ein Signal blind weiterzuleiten, könnte es einer Route durch den Neighborhood-Graphen folgen. Zum Beispiel:

```
NH01
|
▼
NH04
|
▼
NH08
|
▼
NH10
```

Die berechnete Route könnte im Signal gespeichert werden:

```
{
  "route": [
    "NH01",
    "NH04",
```

```
"NH08",  
"NH10"  
]  
}
```

Während der Weiterleitung würde das Signal der geplanten Route folgen.

Mehrere Routingstrategien

Unterschiedliche Experimente können unterschiedliches Routingverhalten erfordern. Mögliche Strategien sind:

- kürzester Pfad
- niedrigste Latenz
- niedrigste Propagation-Kosten
- höchste Bandbreite
- höchste Zuverlässigkeit
- minimale Hop-Anzahl

Die Routingstrategie könnte daher Bestandteil des Signals werden:

```
{  
  "routingStrategy": "lowestLatency"  
}
```

Verbindungsmetadaten

Verbindungen zwischen Neighborhoods können zusätzliche Informationen enthalten, zum Beispiel:

```
{  
  "targetNeighborhoodId": "NH05",  
  "latency": 12,  
  "bandwidth": 100,  
  "packetLoss": 0.01,  
  "cost": 2  
}
```

Routingentscheidungen könnten die Verbindungsqualität berücksichtigen, anstatt alle Verbindungen gleich zu behandeln.

Dynamische Neuberechnung der Route

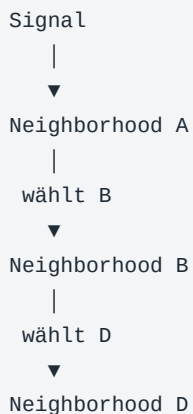
Experimente sind dynamisch. Verbindungen können sich ändern, während ein Signal unterwegs ist. Beispiele dafür sind:

- ein Neighborhood ist nicht verfügbar
- ein Server ist offline
- hohe Latenz
- Überlastung
- ein vorübergehender Verbindungsfehler

Anstatt sofort fehlschlagen, könnte das Signal von seiner aktuellen Position aus eine neue Route berechnen.

Zukünftiges verteiltes Routing

Eine fortgeschrittenere Variante könnte darauf verzichten, den vollständigen Pfad bereits zu Beginn zu berechnen. Stattdessen würde jedes Neighborhood anhand lokaler Informationen nur den nächsten Hop auswählen:



Das ähnelt dem Routing in verteilten Computernetzwerken und könnte adaptives Verhalten ermöglichen.

Mögliche Erweiterungen für Signal

```
{  
  "targetNeighborhoodId": "...",  
  "targetAgentId": "...",  
  "route": [],  
  "currentHop": 0,
```

```
"routingStrategy": "shortest",
"routeCalculated": true
}
```

Mögliche Erweiterungen für Neighborhood

```
{
  "neighbors": {
    "north": "...",
    "south": "...",
    "east": "...",
    "west": "..."
  },
  "connections": [
    {
      "targetNeighborhoodId": "...",
      "latency": 15,
      "cost": 2,
      "bandwidth": 100
    }
  ]
}
```

Mögliche Routingalgorithmen

Welcher Algorithmus am besten geeignet ist, hängt von den verfügbaren Informationen ab.

Algorithmus	Besonders geeignet für
Breadth-First Search (BFS)	Möglichst wenige Neighborhood-Hops
Dijkstra	Gewichtetes Routing (Latenz, Kosten)
A*	Räumliche Neighborhood-Anordnungen mit Koordinaten
Bellman-Ford	Dynamische Kantengewichte
Link-State Routing	Große verteilte Experimente
Reinforcement Learning	Zukünftiges adaptives Schwarm-Routing

Für den aktuellen Prototyp gilt:

- BFS genügt für Routing über den kürzesten Pfad.
- Dijkstra wird sinnvoll, sobald gewichtete Verbindungen (Latenz, Kosten, Bandbreite) eingeführt werden.

Bezug zum bestehenden Neighborhood-Vorschlag

Dieser Vorschlag baut unmittelbar auf der bestehenden Neighborhood-Architektur auf. Der vorherige Vorschlag führte Folgendes ein:

- einen Agent pro Neighborhood
- eindeutige Neighborhood-Zugehörigkeit
- explizite Verbindungen zwischen Neighborhoods
- eine graphbasierte Neighborhood-Topologie

Dieser Vorschlag erweitert die Architektur, indem Signals diese Neighborhood-Verbindungen intelligent nutzen können. Der bestehende Graph könnte als Routing-Infrastruktur dienen. Das implementierungsbezogene [Signal-Modell](#) und [Neighborhood-Modell](#) liefern relevanten Workspace-Kontext. Das [Modul zur Pfadsuche](#) ist derzeit leer; das hier beschriebene Routing ist vorgeschlagen und nicht implementiert.

Langfristige Vision

Intelligentes Routing könnte den Weg für fortgeschrittenes Schwarmverhalten ebnen, darunter:

- adaptives Routing
- Vermeidung von Überlastung
- selbstheilende Kommunikation
- verteilte Pfadermittlung
- kooperatives Routing zwischen Neighborhoods
- dezentrale Entscheidungsfindung
- Lastverteilung
- vorausschauendes Routing
- autonome Optimierung
- emergente Kommunikationsmuster

Diese Fähigkeiten könnten die Local Intelligence Layer über deterministische Signalweiterleitung hinaus zu einem verteilten System führen, das seine Kommunikation selbst organisiert.

Erwartete Vorteile

Im Vergleich zu einfacher Weiterleitung könnte intelligentes Routing Folgendes bieten:

- weniger Propagation-Events
- geringeren Rechenaufwand
- weniger Netzwerkverkehr
- schnellere Signalzustellung
- bessere Skalierbarkeit
- höhere Fehlertoleranz
- realistischere Schwarmkommunikation
- eine Grundlage für kollektive Intelligenz
- eine Grundlage für emergentes Verhalten
- eine Grundlage für zukünftige selbstorganisierende Algorithmen

Autonomiestrategie

Dieses Dokument beschreibt den im Prototyp vorhandenen Autonomieablauf. Es handelt sich um eine Übersicht der Implementierung, nicht um einen Vorschlag für ein künftiges Subsystem.

Funktionsablauf

<code>agentAutonomy()</code>	→ Entscheidung treffen
<code>executeAutonomousAction()</code>	→ Entscheidung ausführen
<code>runAutonomy()</code>	→ geeignete Agenten verarbeiten
<code>autonomyScheduler()</code>	→ einen oder mehrere Simulationstakte ausführen

Aufgaben

- `agentAutonomy()` wertet die lokale Nachbarschaft eines Agenten aus und liefert eine Entscheidung wie `activate`, `listen`, `wait`, `idle` oder `observe`. Die aktuellen Entscheidungen beruhen auf Anzahl und Zustand der lokalen Nachbarn.

- `runAutonomy()` überspringt Agenten, die offline sind, keiner Nachbarschaft angehören oder deren Autonomie ausgesetzt ist. Für jeden geeigneten Agenten analysiert die Funktion die Nachbarschaft, protokolliert eine Beobachtung und übergibt die Entscheidung an die ausführende Funktion.
- `executeAutonomousAction()` wendet unterstützte Zustandsänderungen an, protokolliert sie und wertet Regeln für Zustandsänderungen aus. Eine Aktivierung kann außerdem ein Signal des Typs `autonomous_activation` an einen inaktiven lokalen Nachbarn erzeugen.
- `autonomyScheduler()` führt die angeforderte Anzahl an Simulationstakten aus. Ein Takt analysiert auch Signale und verarbeitet fällige geplante Aktionen; der Scheduler ist selbst kein dauerhaft laufender Hintergrund-Timer.

Die Implementierung befindet sich im Projekt unter `local-intelligence/services/autonomy/` und `local-intelligence/services/simulation/`.

Verwandte Dokumentation

- [Entitätenkarte](#)
- [Prototyp: Datenbank und Datenmodell](#)
- [Beispiel: Signal, Autonomie und Folgeaktion](#)

Beispiel: Signal, Autonomie und Folge-Propagation

Status: Die hier verwendeten Autonomie- und Verzögerungsmechanismen sind im Prototyp vorhanden. Ablauf und Regelkonfiguration sind jedoch beispielhaft. Es wird weder behauptet, dass genau dieser Gesamtprozess bereits ausgeführt wurde, noch dass jedes Detail der Signalweiterleitung durch die aktuelle Auswertungslogik garantiert ist.

Dieses Beispiel beschreibt einen möglichen Ablauf in einem 3×3-Agentenfeld. Es dient ausschließlich der Dokumentation und ändert weder Laufzeitdaten noch die Rule-Konfiguration.

Ausgangszustand

X bedeutet inaktiv, A bedeutet aktiv.

```
+-----+-----+-----+
```

```
| 1 X | 2 X | 3 X |  
+-----+-----+-----+  
| 4 X | 5 A | 6 X |  
+-----+-----+-----+  
| 7 X | 8 X | 9 X |  
+-----+-----+-----+
```

Agent 5 ist aktiv; alle anderen Agenten sind inaktiv.

Beteiligte Agenten

Agent 5: aktiver Sender der ersten Signale
Agent 2: Ziel eines Aktivierungssignals
Agent 4: Ziel eines Aktivierungssignals
Agent 1: wird durch einen Autonomie-Tick aktiv
Agent 9: Ziel des Folge-Signals von Agent 1

Phase 1: Agent 5 sendet Aktivierungssignale

Agent 5 sendet zwei Signale des Typs activate:

```
{  
  type: "activate",  
  sourceAgentId: "agent-5",  
  targetAgentId: "agent-2",  
  payload: {  
    strength: 1,  
  },  
  properties: {  
    ttl: 10,  
    propagationMode: "unicast",  
    propagationScope: "neighborhood",  
  },  
}  
  
{  
  type: "activate",  
  sourceAgentId: "agent-5",  
  targetAgentId: "agent-4",  
  payload: {  
    strength: 1,  
  },  
  properties: {  
    ttl: 10,  
    propagationMode: "unicast",  
    propagationScope: "neighborhood",  
  },  
}
```

```
},  
}
```

Eine beispielhafte Rule-Konfiguration:

```
{  
  id: "rule-activate-agent",  
  enabled: true,  
  signalType: "activate",  
  scope: "global",  
  action: "activate",  
  threshold: 1,  
}
```

Wenn die Signale angenommen werden und die Rule greift, werden Agent 2 und Agent 4 aktiv:

```
+-----+-----+-----+  
| 1 X | 2 A | 3 X |  
+-----+-----+-----+  
| 4 A | 5 A | 6 X |  
+-----+-----+-----+  
| 7 X | 8 X | 9 X |  
+-----+-----+-----+
```

Konzeptioneller Ablauf:

```
Agent 5  
-> activate-Signal an Agent 2  
  -> signalEngine  
    -> Neighborhood-Analyse  
    -> Rule-Auswertung  
    -> activate-Rule  
    -> Agent 2 wird aktiv  
    -> Protokollierung
```

```
Agent 5  
-> activate-Signal an Agent 4  
  -> signalEngine  
    -> Neighborhood-Analyse  
    -> Rule-Auswertung  
    -> activate-Rule  
    -> Agent 4 wird aktiv  
    -> Protokollierung
```

Phase 2: Ein Autonomie-Tick aktiviert Agent 1

Im dargestellten Feld hat Agent 1 zwei aktive lokale Nachbarn (Agent 2 und Agent 4). Die aktuelle Autonomielogik kann deshalb für einen geeigneten inaktiven Agenten `activate` mit dem Grund `high_local_activity` zurückgeben.

Die Entscheidung hat folgende Form:

```
{
  agentId: "agent-1",
  action: "activate",
  reason: "high_local_activity",
}
```

Der Tick führt die Entscheidung aus und wertet Regeln für Zustandsänderungen aus:

```
Autonomie-Tick
-> Die Nachbarschaft von Agent 1 wird analysiert
-> agentAutonomy entscheidet: activate
-> executeAutonomousAction
-> Agent 1 wechselt von inactive zu active
-> state_changed-Trigger werden ausgewertet
```

Danach sieht das Feld so aus:

```
+-----+-----+-----+
| 1 A | 2 A | 3 X |
+-----+-----+-----+
| 4 A | 5 A | 6 X |
+-----+-----+-----+
| 7 X | 8 X | 9 X |
+-----+-----+-----+
```

Phase 3: Agent 1 löst eine Folge-Rule aus

Agent 1 besitzt eine beispielhafte Rule, die auf den Zustandswechsel `inactive -> active` reagiert:

```
{
  id: "rule-agent-1-follow-up",
  enabled: true,
  scope: "agent",
  agentId: "agent-1",
  trigger: {
    type: "state_changed",
    fromState: "inactive",
  }
}
```

```

    toState: "active",
  },
  action: "send_signal",
  appendedSignal: {
    delayMs: 5000,
    targetAgentId: "agent-9",
    signalType: "deactivate",
    signalPayload: {
      strength: 1,
      reason: "agent-1-became-active",
    },
  },
  signalProperties: {
    ttl: 10,
    propagationMode: "broadcast",
    propagationScope: "all",
  },
},
},
}

```

Der Handler für Zustandsänderungen plant die Aktion, statt das Signal sofort zu senden:

```

Agent 1 wird aktiv
-> state_changed-Trigger passt
-> appendedSignal wird als pending gespeichert
-> 5000 ms warten
-> ein späterer Simulationstakt prüft executeAt
-> ein Signal an Agent 9 wird erzeugt

```

Ein interner Eintrag für die geplante Aktion kann beispielsweise so aussehen:

```

{
  action: "send_signal",
  sourceAgentId: "agent-1",
  targetAgentId: "agent-9",
  signalType: "deactivate",
  signalPayload: {
    strength: 1,
    reason: "agent-1-became-active",
  },
},
signalProperties: {
  ttl: 10,
  propagationMode: "broadcast",
  propagationScope: "all",
},
status: "pending",
executeAt: "2026-09-22T12:00:05.000Z",
}

```

Ein Simulationstakt verarbeitet die geplante Aktion nach ihrem Fälligkeitszeitpunkt; die Verzögerung wird nicht durch einen permanenten Hintergrund-Timer umgesetzt.

Phase 4: Agent 9 und weitere Empfänger werden inaktiv

Für das gewünschte Ergebnis benötigt das aktive RuleSet eine Rule, die den Empfänger inaktiv setzt, sowie eine Propagation-Rule, die das Signal weiterleitet. Die folgenden Beispiele beschreiben die beabsichtigte Konfiguration:

Rule zum Inaktivieren

```
{
  id: "rule-deactivate-agent",
  enabled: true,
  signalType: "deactivate",
  scope: "global",
  action: "inactivate",
  threshold: 1,
}
```

Diese Rule soll den empfangenden Agenten auf `inactive` setzen.

Propagation-Rule

```
{
  id: "rule-propagate-deactivate",
  enabled: true,
  signalType: "deactivate",
  scope: "global",
  action: "propagate",
  threshold: 1,
}
```

Diese Rule soll das Signal an weitere erreichbare Agenten weiterleiten. Beide Rules müssen im aktiven RuleSet enthalten sein. Das Beispiel setzt voraus, dass die Auswertungslogik bei jedem Empfänger zuerst das Inaktivieren und danach die Weiterleitung verarbeitet. Reihenfolge, Zielauswahl und Abbruchverhalten müssen anhand der konfigurierten Propagation-Implementierung geprüft werden.

Beabsichtigter Ablauf je Empfänger:

```
deactivate-Signal
-> inactivate-Rule
-> Empfänger wird inaktiv
-> propagate-Rule
-> Signal wird an weitere erreichbare Agenten gesendet
```

Nachdem Agent 9 das Signal empfangen und verarbeitet hat, ist das Feld so darzustellen:

```

+-----+-----+-----+
| 1 A | 2 A | 3 X |
+-----+-----+-----+
| 4 A | 5 A | 6 X |
+-----+-----+-----+
| 7 X | 8 X | 9 X |
+-----+-----+-----+

```

Agent 9 bleibt in diesem dargestellten Schritt inaktiv; das Signal verursacht bei diesem Agent keine zusätzliche Zustandsänderung. Weitere Agenten ändern ihren Zustand nur, wenn die Propagation sie erreicht und die Inaktivierungs-Rule greift.

Vollständiger beispielhafter Ablauf

Ausgangszustand:

```

1X 2X 3X
4X 5A 6X
7X 8X 9X

```

1. Agent 5 sendet activate an Agent 2.
2. Die activate-Rule aktiviert Agent 2.
3. Agent 5 sendet activate an Agent 4.
4. Die activate-Rule aktiviert Agent 4.
5. Ein Autonomie-Tick entscheidet activate für Agent 1.
6. Agent 1 wird aktiv.
7. Der Trigger state_changed inactive -> active passt auf die Rule von Agent 1.
8. Die Rule plant ein appendedSignal mit delayMs 5000.
9. Ein späterer Simulationstakt erzeugt deactivate von Agent 1 an Agent 9.
10. Die deactivate -> inactivate-Rule setzt Agent 9 auf inaktiv.
11. Die deactivate -> propagate-Rule leitet das Signal weiter, sofern die aktive Konfiguration dies unterstützt.
12. Jeder weitere Empfänger wird nur dann inaktiv, wenn er das Signal erhält und die Inaktivierungs-Rule greift.

Technische Voraussetzungen und Hinweise

- Agent 5 benötigt zwei gültige Zielagenten; die beiden Signale werden separat gesendet.
- Agent 1 muss tatsächlich von inactive zu active wechseln, damit der Zustandsänderungs-Trigger greift.
- Agent 1 benötigt eine Neighborhood für das dargestellte Verhalten auf Basis lokaler Nachbarn.
- Agent 9 muss existieren, damit createSignal das Folge-Signal erzeugen kann.
- Die Agenten müssen mit dem aktiven Propagation-Modus, Geltungsbereich und der Neighborhood-Konfiguration erreichbar sein.

- Die Verhaltensweisen deactivate -> inactivate und deactivate -> propagate müssen im aktiven RuleSet unterstützt und konfiguriert sein; das Beispiel bestätigt nicht die genaue Auswertungsreihenfolge.
- Die geplante Aktion wird durch einen Simulationstakt verarbeitet, nachdem 5000 ms vergangen sind, nicht durch einen permanenten Hintergrund-Timer.

Siehe auch [Autonomiestrategie](#) und [Entitätenkarte](#).

Beispiel: Crows und Holy – interaktive Show-Aktivierung

Status: Anwendungsidee. Dieses Beispiel beschreibt ein mögliches Erlebnis mit dem Local Intelligence Layer. Es ist kein Nachweis dafür, dass die beschriebene Smartphone-Erkennung, Signalorchestrierung oder Show-Visualisierung bereits implementiert ist.

Ziel

Crows könnte den Local Intelligence Layer bei einer Live-Show und für kommerzielle Markenaktivierungen einsetzen. Als Beispiel dient Holy, ein Anbieter von Getränkepulver, das in Wasser gemischt und geschüttelt wird, um ein Getränk herzustellen.

Die Bewegung, die zum Produkt passt – das Schütteln eines Shakers – wird dabei zur gemeinsamen Publikums-Choreografie. Das Publikum löst die Aktion selbst aus; die Software koordiniert die Weitergabe und macht den Fortschritt für die Show sichtbar.

Aufbau: 3 × 3 Neighborhoods mit je 3 × 3 Agents

Die Bühne oder die Publikumsanzeige zeigt ein Raster aus neun Neighborhoods. Jedes Neighborhood enthält neun Agents. Die drei Geschmacksgruppen **Kirsche**, **Waldmeister** und **Wassermelone** belegen jeweils eine Spalte. Die drei Neighborhoods einer Spalte bilden die drei Ebenen der Flasche – von unten nach oben.

	SHOW-DISPLAY		
	Kirsche	Waldmeister	Wassermelone
Oben	[NH]	[NH]	[NH]
Mitte	[NH]	[NH]	[NH]
Unten	[NH]	[NH]	[NH]
	↑	↑	↑

Jedes [NH] enthält ein 3×3 -Agent-Raster. Die Zuordnung von Agents zu Personen, Geräten oder virtuellen Teilnehmern hängt von der konkreten Show-Implementierung ab.

Ablauf der Aktivierung

1. **Startsignal:** Die untersten Neighborhoods werden aktiviert. Die Agents dort fordern die zugehörigen Teilnehmenden auf, ihr Smartphone wie einen Shaker zu schütteln.
2. **Bewegung erkennen:** Eine App oder ein angeschlossener Erkennungsdienst prüft ein vereinbartes Kriterium, zum Beispiel eine Mindestanzahl von Schüttelbewegungen oder eine erkannte Bewegungsintensität innerhalb eines Zeitfensters.
3. **Schwelle erreicht:** Sobald der Grenzwert erreicht ist, streckt die teilnehmende Person ihr Smartphone nach oben. Diese Geste bestätigt den Abschluss und löst ein Signal des Agents aus.
4. **Signal weitergeben:** Das Signal aktiviert den passend verbundenen Agent im Neighborhood darüber. Dieser beginnt den nächsten Abschnitt der Choreografie und fordert die dortigen Teilnehmenden zum Schütteln auf.
5. **Füllstand darstellen:** Die Aktivierung steigt Ebene für Ebene nach oben. Auf dem Show-Display leuchten die Agents oder Neighborhoods entsprechend auf; aus der Vogelperspektive sieht es aus, als würde sich eine Holy-Flasche füllen.
6. **Abschluss:** Wenn die oberste Ebene erreicht ist, kann eine gemeinsame Animation, ein Jingle oder ein Call-to-action ausgelöst werden.

Die drei Geschmacksspalten können unabhängig oder synchron laufen. So lässt sich beispielsweise beobachten, welche Gruppe zuerst die Flasche füllt, oder die Show kann alle Gruppen auf ein gemeinsames Finale hin synchronisieren.

Was der Local Intelligence Layer beiträgt

- **Lokale Reaktionen:** Jeder Agent reagiert auf ein lokales Ereignis, statt dass ein zentraler Ablauf jede einzelne Person direkt steuern muss.
- **Signalpropagation:** Ein bestätigter Fortschritt kann gezielt an die nächste Ebene weitergereicht werden.
- **Konfigurierbare Regeln:** Schüttel-Schwelle, Zeitfenster, Wiederholungen und Übergangsbedingungen können als Regeln für ein Experiment festgelegt werden.
- **Live-Visualisierung:** Aggregierte Zustände der Agents und Neighborhoods können den Füllstand und den Fortschritt der Gruppen darstellen.

- **Wiederverwendbarkeit:** Das gleiche Muster kann für andere Produkte, Marken, Bühnenbilder und Publikumsaktionen angepasst werden.

Kommerzielle Nutzung und Erfolgsmessung

Als Markenaktivierung kann das Erlebnis eine Produktbotschaft in eine gemeinschaftliche Aktion übersetzen. Mögliche, aggregierte Kennzahlen sind zum Beispiel:

- Anteil der aktivierten Teilnehmenden, die die Aufgabe abschließen
- Zeit bis zum Erreichen jeder Ebene und der gesamten Flasche
- Abschlussrate je Geschmacksgruppe
- Anzahl der ausgelösten Signale und wiederholte Teilnahme
- Reaktion auf unterschiedliche Schwellen oder Show-Abläufe

Diese Kennzahlen können Crows und dem Markenpartner helfen, verschiedene Choreografien oder Aktivierungsvarianten zu vergleichen. Sie sollten nur erhoben werden, wenn die Erhebung transparent erklärt und für den Zweck erforderlich ist.

Datenschutz, Sicherheit und Zugänglichkeit

- Die Teilnahme sollte freiwillig sein und vor der Aktivierung verständlich erklärt werden.
- Wenn die Bewegungserkennung über Smartphones erfolgt, sollte die App nur die für die Schwelle nötigen Bewegungsdaten verarbeiten. Eine Speicherung von Rohdaten oder eine Zuordnung zu einer identifizierbaren Person ist für die aggregierte Show-Visualisierung nicht erforderlich.
- Grenzwerte und Zeitfenster sollten so gewählt werden, dass die Bewegung sicher und für unterschiedliche Fähigkeiten anpassbar ist. Eine alternative Geste oder manuelle Bestätigung kann Teilnahme ohne Schütteln ermöglichen.
- Die Smartphone-Geste sollte mit ausreichend Abstand und ohne Werfen oder gefährliches Schwingen ausgeführt werden.
- Die Show sollte auch dann fortgesetzt werden können, wenn einzelne Geräte offline sind oder Teilnehmende nicht mitmachen.

Umsetzungsgrenze

Der Local Intelligence Layer könnte die Agent- und Neighborhood-Zustände, Regeln sowie die Signalweitergabe orchestrieren. Die Erkennung von Smartphone-Bewegungen, die Verbindung zu einem Show-Display und die Echtzeitdarstellung benötigen passende App-, Hardware- oder Integrationskomponenten. Diese Schnittstellen und die tatsächliche Performance müssten vor einem Live-Einsatz separat validiert werden.